

// File: Sources/LingXiCore/Modules/Agent/AgentRuntime.swift

```

import Foundation
import LingXiProtocol

/// Agent 最小编排入口（Session 化）。
/// sendMessage: append user → SessionContextBuilder 构建模型输入 → 推理。
/// 高频 delta 只走 DMA + 本轮内存 buffer 聚合；
/// Session 在 turn 完成时一次性写入最终 assistant content。
/// 本类型不解析任何 Provider JSON。
public actor AgentRuntime {
    private let store: any SessionStore
    private let contextEngine: L1ContextEngine
    private let modelBus: ModelBus
    private let dataPlane: DataPlane
    private let toolRuntime: ToolRuntime
    private let questions: QuestionRuntime
    private let permissions: PermissionEngine
    private let performanceStore: PerformanceStore
    private let contextPager: ContextPager
    private let projectScanner: ProjectScanner
    private let compactor: ContextCompactor
    private let budgetPlanner: ContextBudgetPlanner
    private let persistence: SQLitePersistenceStore?
    private let eventSink: @Sendable (CoreEvent) async -> Void
    private let interactive: Bool
    private let diagnosticsEnabled: Bool
    private var runtimes: [SessionID: SessionRuntime] = [:]
    private let modelResolver: SubagentModelResolver
    private let scheduler: AgentRunScheduler
    private let limits: SubagentRuntimeLimits
    private var behaviorProfile: AgentBehaviorProfile
    private var behaviorInstructionsEnabled: Bool
    private let behaviorSystemContext: @Sendable (AgentBehaviorProfile, SubagentExecutionProfile?) -> String?
    private let deadlinePolicy: ExecutionDeadlinePolicy
    private let restoreScheduler: SessionRestoreScheduler?
    private let diagnostics: RuntimeDiagnosticsStore?
    private var runs: [AgentRunID: AgentRunInfo] = [:]
    private var results: [AgentRunID: SubagentResult] = [:]
    private var executionProfiles: [AgentRunID: SubagentExecutionProfile] = [:]
    private var activeSessions: Set<SessionID> = []
    private var runDeadlines: [AgentRunID: ExecutionDeadline] = [:]
    private var resultWaiters: [AgentRunID: [CheckedContinuation<SubagentResult, Error>]] = [:]
    private var shuttingDown = false
    private let cacheController: ContextCacheController
    private let maxAgentLoopSteps: Int

    init(
        store: any SessionStore,
        contextEngine: L1ContextEngine,
        modelBus: ModelBus,
        dataPlane: DataPlane,
        toolRuntime: ToolRuntime,
        questions: QuestionRuntime,
        permissions: PermissionEngine,
        performanceStore: PerformanceStore,
        contextPager: ContextPager,
        projectScanner: ProjectScanner,
        eventSink: @escaping @Sendable (CoreEvent) async -> Void,
        compactor: ContextCompactor = ContextCompactor(),
        budgetPlanner: ContextBudgetPlanner = ContextBudgetPlanner(),
        persistence: SQLitePersistenceStore? = nil,
        cacheController: ContextCacheController? = nil,
        interactive: Bool = false,
        diagnosticsEnabled: Bool = false,
        modelResolver: SubagentModelResolver,

```

```

limits: SubagentRuntimeLimits = SubagentRuntimeLimits(),
scheduler: AgentRunScheduler? = nil,
behaviorProfile: AgentBehaviorProfile = .build,
behaviorInstructionsEnabled: Bool = false,
behaviorSystemContext: @escaping @Sendable (AgentBehaviorProfile, SubagentExecutionProfile?) -> String? = { _, _ in nil },
maxAgentLoopSteps: Int = 32,
deadlinePolicy: ExecutionDeadlinePolicy = ExecutionDeadlinePolicy(),
restoreScheduler: SessionRestoreScheduler? = nil,
diagnostics: RuntimeDiagnosticsStore? = nil
) {
    self.store = store
    self.contextEngine = contextEngine
    self.modelBus = modelBus
    self.dataPlane = dataPlane
    self.toolRuntime = toolRuntime
    self.questions = questions
    self.permissions = permissions
    self.performanceStore = performanceStore
    self.contextPager = contextPager
    self.projectScanner = projectScanner
    self.eventSink = eventSink
    self.compactor = compactor
    self.cacheController = cacheController ?? ContextCacheController(contextPager: contextPager, scanner: projectScanner, compactor:
↳ compactor)
    self.budgetPlanner = budgetPlanner
    self.persistence = persistence
    self.interactive = interactive
    self.diagnosticsEnabled = diagnosticsEnabled
    self.modelResolver = modelResolver
    self.limits = limits
    self.scheduler = scheduler ?? AgentRunScheduler(limits: limits)
    self.behaviorProfile = behaviorProfile
    self.behaviorInstructionsEnabled = behaviorInstructionsEnabled
    self.behaviorSystemContext = behaviorSystemContext
    self.maxAgentLoopSteps = maxAgentLoopSteps
    self.deadlinePolicy = deadlinePolicy
    self.restoreScheduler = restoreScheduler
    self.diagnostics = diagnostics
}

// MARK: - Session 生命周期

public func createSession() async throws -> SessionID {
    let session = try await store.create(kind: .primary, parentSessionID: nil, rootSessionID: nil, spawnedByRunID: nil,
↳ spawnedByToolCallID: nil, title: nil)
    await cacheController.resetSession(session.id)
    runtimes[session.id] = makeRuntime(for: session.id)
    await eventSink(.sessionCreated(session.id))
    return session.id
}

public func restore() async throws {
    guard !shuttingDown else { return }
    try await compactor.restoreDerived()
    guard !shuttingDown else { return }
    guard let persistence else { return }
    let persistedRuns = try await persistence.loadAgentRuns()
    for persisted in persistedRuns {
        var run = persisted
        if !run.status.isTerminal {
            let calls = try await persistence.toolBatches(sessionID: run.sessionID).flatMap(\.toolCallStates).filter { $0.provenance.
↳ agentRunID == run.runID }
            if !calls.isEmpty {
                let waiting = calls.contains { $0.state == .waitingForHuman || ($0.request != nil && $0.reply == nil) }
                run = AgentRunInfo(runID: run.runID, sessionID: run.sessionID, projectID: run.projectID, parentRunID: run.parentRunID,

```

```

↳ rootRunID: run.rootRunID, agentKind: run.agentKind, status: waiting ? .waitingForUser : .recoveryRequired, modelSelection: run.
↳ modelSelection, startedAt: run.startedAt, latestActivityAt: .now, usage: run.usage, title: run.title)
    if waiting {
        activeSessions.insert(run.sessionID)
    }
    try await persistence.saveAgentRun(run)
} else {
    run = AgentRunInfo(runID: run.runID, sessionID: run.sessionID, projectID: run.projectID, parentRunID: run.parentRunID,
↳ rootRunID: run.rootRunID, agentKind: run.agentKind, status: .recoveryRequired, modelSelection: run.modelSelection, startedAt: run.
↳ startedAt, finishedAt: .now, latestActivityAt: .now, error: CoreError(code: .toolCancelled, message: "Core 重启, 运行需要恢复"),
↳ usage: run.usage, title: run.title)
    try await persistence.saveAgentRun(run)
}
}
runs[run.runID] = run
if let profile = try await persistence.agentRunProfile(run.runID) { executionProfiles[run.runID] = profile }
if let result = try await persistence.agentRunResult(run.runID) { results[run.runID] = result }
}
for run in runs.values where run.status == .waitingForUser {
    guard !shuttingDown else { return }
    do {
        _ = try await runtime(for: run.sessionID, run: run)
    } catch {
        logDiagnostic("Failed to restore runtime for run=\(run.runID.rawValue) session=\(run.sessionID.rawValue): \((error)")
        activeSessions.remove(run.sessionID)
        let failedRun = AgentRunInfo(
            runID: run.runID,
            sessionID: run.sessionID,
            projectID: run.projectID,
            parentRunID: run.parentRunID,
            rootRunID: run.rootRunID,
            agentKind: run.agentKind,
            status: .recoveryRequired,
            modelSelection: run.modelSelection,
            startedAt: run.startedAt,
            finishedAt: .now,
            latestActivityAt: .now,
            error: (error as? CoreError) ?? CoreError(code: .provider, message: "恢复会话模型失败: \((error.localizedDescription)"),
            usage: run.usage,
            title: run.title
        )
        runs[run.runID] = failedRun
        try? await persistence.saveAgentRun(failedRun)
    }
}
}
await restoreScheduler?.markReady()
}

public func shutdown() async {
    shuttingDown = true
    for runtime in runtimes.values { await runtime.shutdown() }
}

public func listSessions() async throws -> [SessionInfo] {
    try await store.listSessions().map { $0.toInfo() }
}

public func listChildSessions(_ parentSessionID: SessionID) async throws -> [SessionInfo] {
    try await store.listSessions().filter { $0.parentSessionID == parentSessionID }.map { $0.toInfo() }
}

public func listAgentRuns(_ sessionID: SessionID) -> [AgentRunInfo] {
    runs.values.filter { $0.sessionID == sessionID }.sorted { $0.latestActivityAt < $1.latestActivityAt }
}

```

```

public func allAgentRuns() -> [AgentRunInfo] {
    runs.values.sorted { $0.latestActivityAt < $1.latestActivityAt }
}

public func orphanRunIDs() async -> [AgentRunID] {
    let schedulerState = await scheduler.snapshot()
    let active = Set(schedulerState.active + schedulerState.queued)
    return runs.values.filter {
        !$0.status.isTerminal && $0.status != .waitingForUser && $0.status != .waitingForTool && !active.contains($0.runID)
    }.map(\.runID).sorted { $0.rawValue < $1.rawValue }
}

public func agentRun(_ runID: AgentRunID) throws -> AgentRunInfo {
    guard let run = runs[runID] else { throw CoreError(code: .agentRunNotFound, message: "AgentRun 不存在: \(runID.rawValue)") }
    return run
}

public func agentRun(_ runID: AgentRunID, requester: AgentRunID) throws -> AgentRunInfo {
    let run = try agentRun(runID)
    try requireSameTree(run, requester: requester)
    return run
}

public func resumeAgentRun(_ runID: AgentRunID) async throws -> AgentRunInfo {
    let run = try agentRun(runID)
    guard !run.status.isTerminal else { return run }
    guard !shuttingDown else { throw CoreError(code: .notReady, message: "Agent Runtime 正在关闭") }
    _ = try await runtime(for: run.sessionID, run: run)
    return try agentRun(runID)
}

public func agentRunResult(_ runID: AgentRunID) throws -> SubagentResult {
    guard let result = results[runID] else { throw CoreError(code: .agentRunNotFound, message: "AgentRun 尚无稳定结果: \(runID.rawValue)
    ⇐ ") }
    return result
}

public func agentRunResult(_ runID: AgentRunID, requester: AgentRunID) throws -> SubagentResult {
    let run = try agentRun(runID)
    try requireSameTree(run, requester: requester)
    return try agentRunResult(runID)
}

public func waitForAgentRunResult(_ runID: AgentRunID) async throws -> SubagentResult {
    if let result = results[runID] { return result }
    _ = try agentRun(runID)
    return try await withCheckedThrowingContinuation { continuation in
        resultWaiters[runID, default: []].append(continuation)
    }
}

/// Workflows share spawn, deadline, model/profile resolution, and the canonical child Run scheduler.
public func makeWorkflowRuntime() -> WorkflowRuntime {
    WorkflowRuntime(persistence: persistence, executor: { [weak self] _, task, parent, started in
        guard let self else { return WorkflowTaskCompletion(error: CoreError(code: .notReady, message: "Agent 未就绪")) }
        do {
            let rendered = [task.role.map { "Role: \"($0)\" }, task.instructions.map { "Instructions: \"($0)\" }, task.context.map { "
            ⇐ Context: \"($0)\" }, task.task].compactMap { $0 }.joined(separator: "\n\n")
            let (childSessionID, run) = try await self.spawn(parentSessionID: parent.parentSessionID, parentRunID: parent.parentRunID,
            ⇐ task: rendered, title: task.title, modelSelection: task.modelSelection, profile: task.executionProfile)
            await started(WorkflowTaskProvenance(parentSessionID: parent.parentSessionID, parentRunID: parent.parentRunID,
            ⇐ childSessionID: childSessionID, childRunID: run.runID))
            return WorkflowTaskCompletion(result: try await self.waitForAgentRunResult(run.runID))
        } catch let error as CoreError {
            return WorkflowTaskCompletion(error: error)
        }
    })
}

```

```

        } catch {
            return WorkflowTaskCompletion(error: CoreError(code: .provider, message: String(describing: error)))
        }
    }, diagnostics: diagnostics)
}

public func agentTree(_ rootSessionID: SessionID) async throws -> AgentTreeNode {
    let sessions = try await store.listSessions()
    func node(_ session: Session) -> AgentTreeNode {
        let latest = runs.values.filter { $0.sessionID == session.id }.max { $0.latestActivityAt < $1.latestActivityAt }
        return AgentTreeNode(session: session.toInfo(), latestRun: latest, children: sessions.filter { $0.parentSessionID == session.id }
        ↪ ).map(node))
    }
    guard let root = sessions.first(where: { $0.id == rootSessionID }) else { throw CoreError(code: .sessionNotFound, message: "Session
    ↪ 不存在") }
    return node(root)
}

public func sessionSnapshot(_ id: SessionID) async throws -> LegacySessionSnapshot {
    try await store.session(id).toSnapshot()
}

public func renameSession(_ id: SessionID, title: String?) async throws -> SessionInfo {
    let normalized = title?.trimmingCharacters(in: .whitespacesAndNewlines)
    return try await store.updateTitle(id, title: normalized?.isEmpty == true ? nil : normalized).toInfo()
}

public func selectModel(_ selection: ModelSelection, assembly: ModelRuntimeAssembly? = nil) async throws {
    try await modelResolver.setDefaultSelection(selection, assembly: assembly)
}

public func contextSnapshot(_ id: SessionID) async -> L1ContextSnapshot? {
    await contextEngine.latestSnapshot(for: id)
}

public func ensureContextSnapshot(_ id: SessionID) async throws -> L1ContextSnapshot {
    if let snapshot = await contextEngine.latestSnapshot(for: id) { return snapshot }
    let session = try await store.session(id)
    let entries = await contextEngine.entries(for: session)
    let estimatedTokens = ConservativeTokenEstimator().estimate(entries: entries)
    return await contextEngine.snapshot(for: session, activeEntries: entries, estimatedTokens: estimatedTokens)
}

public func performance(_ id: SessionID) async -> TurnPerformanceReport? {
    await performanceStore.report(for: id)
}

public func currentBehaviorProfile() -> AgentBehaviorProfile { behaviorProfile }

public func setBehaviorProfile(_ profile: AgentBehaviorProfile) {
    behaviorProfile = profile
    behaviorInstructionsEnabled = true
}

public func projectCache(sessionID: SessionID? = nil) async -> ProjectCacheDebugSnapshot {
    let metrics = await contextPager.debugMetrics(projectRoot: projectScanner.root)
    let derived = if let sessionID {
        await compactor.cacheMetrics(sessionID: sessionID)
    } else {
        await compactor.cacheMetrics()
    }
    let sessionL2Pages = if let sessionID {
        (await contextEngine.latestSnapshot(for: sessionID))?.metrics.projectPageCount ?? 0
    } else {
        metrics.l2Pages
    }
}

```

```

    }
    let sessionL2Characters = if let sessionID {
        (await contextEngine.latestSnapshot(for: sessionID))?.metrics.projectCharacterCount ?? 0
    } else {
        metrics.l2Characters
    }
    return ProjectCacheDebugSnapshot(
        l2Pages: sessionL2Pages,
        l2Characters: sessionL2Characters,
        l2HitRate: metrics.l2Lookups == 0 ? nil : Double(metrics.l2Hits) / Double(metrics.l2Lookups),
        l3Pages: metrics.l3Pages,
        staleRebuilds: metrics.staleRebuilds,
        symbolCount: metrics.symbolCount,
        symbolIndexedFiles: metrics.symbolIndexedFiles,
        referenceCount: metrics.referenceCount,
        dependencyCount: metrics.dependencyCount,
        sessionL2DerivedPages: derived.l2Pages,
        derivedL3Pages: derived.l3Pages,
        derivedPageOutCount: derived.pageOutCount,
        derivedPageInCount: derived.pageInCount,
        historicalToolEvidencePages: derived.historicalToolPages,
        derivedL3Hits: derived.l3Hits,
        sessionL2DerivedHits: derived.l2Hits,
        sessionL2DerivedPromotions: derived.l2Promotions
    )
}

public func latestContextManifest(_ sessionID: SessionID) async -> ProviderContextManifest? {
    await runtimes[sessionID]?.latestContextManifest
}

public func compact(_ sessionID: SessionID) async throws -> CompactSessionResponse {
    try await runtime(for: sessionID).compactNow()
}

public func cacheMetrics(_ sessionID: SessionID) async -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int,
    ↪ historicalToolPages: Int, l3Hits: Int, l2Hits: Int, l2Promotions: Int)? {
    await runtimes[sessionID]?.cacheMetrics()
}

public func contextUnitStates(_ sessionID: SessionID) async -> [ContextUnitDebugSnapshot] {
    await runtimes[sessionID]?.contextUnitStates() ?? []
}

// MARK: - 对话

/// 在 Session 中发起一轮对话，返回该轮的 DMA 通道。
public func sendMessage(_ sessionID: SessionID, _ content: String) async throws -> OpenedStream {
    guard !activeSessions.contains(sessionID) else { throw CoreError(code: .turnAlreadyRunning, message: "该 Session 已有进行中的对话轮次
    ↪ ") }
    // Preserve the established contract: an unavailable provider still records the user turn.
    if modelBus.gateway.modelID == nil { return try await runtime(for: sessionID).startTurn(content) }
    do {
        // Reserve before the first await so concurrent callers cannot create a second lane.
        activeSessions.insert(sessionID)
        let session = try await store.session(sessionID)
        let run = try await createRun(session: session, parentRunID: nil, requestedModel: nil, title: session.title, profile:
        ↪ behaviorProfile.executionProfile)
        return try await runtime(for: sessionID, run: run).startTurn(content)
    } catch {
        activeSessions.remove(sessionID)
        throw error
    }
}

```

```

public fun spawn(parentSessionID: SessionID, parentRunID: AgentRunID, task: String, title: String? = nil, modelSelection:
↳ ModelSelection? = nil, profile: SubagentExecutionProfile? = nil, toolCallID: ToolCallID? = nil) async throws -> (SessionID,
↳ AgentRunInfo) {
    let parent = try await store.session(parentSessionID)
    let parentRun = try agentRun(parentRunID)
    guard parentRun.sessionID == parentSessionID, !parentRun.status.isTerminal else { throw CoreError(code: .toolArgumentInvalid,
↳ message: "Parent AgentRun 与 Session 不匹配或已结束") }
    let depth = try await depth(of: parent)
    guard depth < limits.maxSubagentDepth else { throw CoreError(code: .subagentDepthExceeded, message: "Subagent 最大深度已达到") }
    guard runs.values.filter({ $0.rootRunID == parentRun.rootRunID }).count < limits.maxTotalRunsPerRootRun else {
        throw CoreError(code: .subagentDepthExceeded, message: "单个 Root AgentRun 的运行总数已达到上限")
    }
    let normalizedProfile = try normalized(profile: profile, parent: executionProfiles[parentRunID])
    let requestedModel = normalizedProfile?.modelSelection ?? modelSelection
    let resolved = try await modelResolver.resolve(requestedModel, subagent: true)

    let endpointProfile = resolved.assembly.contextProfile
    let requestedWindow = normalizedProfile?.contextProfile.flatMap(Int.init)
    let effectiveWindow = min(endpointProfile.contextWindowTokens, requestedWindow ?? endpointProfile.contextWindowTokens)
    let prospectiveProfile = ModelContextProfile(
        contextWindowTokens: effectiveWindow,
        maxOutputTokens: endpointProfile.maxOutputTokens,
        recommendedOutputReserveTokens: endpointProfile.recommendedOutputReserveTokens,
        source: endpointProfile.source
    )
    let childTools = await toolRuntime.availableDefinitions(sessionID: parentSessionID, interactive: interactive, executionProfile:
↳ normalizedProfile)
    let estimator = ConservativeTokenEstimator()
    let toolTokens = estimator.estimate(tools: childTools)
    let preferred = normalizedProfile?.budgetProfile.flatMap(Int.init)
    let planner = preferred.map { budgetPlanner.with(preferredActiveTokens: $0) } ?? budgetPlanner
    let prospectiveBudget = planner.plan(profile: prospectiveProfile, toolTokens: toolTokens)
    let initialMandatoryTokens = await contextEngine.initialMandatoryTokens(task: task, estimator: estimator)

    guard prospectiveBudget.hardInputLimit >= initialMandatoryTokens, prospectiveBudget.hardInputLimit > 0 else {
        throw CoreError(
            code: .contextProfileNotViable,
            message: "contextProfile 不可行: hardInputLimit=\(prospectiveBudget.hardInputLimit) 不足以容纳初始 mandatory context (\(
↳ initialMandatoryTokens) tokens)"
        )
    }

    let child: Session
    var run: AgentRunInfo
    if let persistence {
        let root = try await persistence.mainRootBinding()
        child = Session(id: SessionID(UUID().uuidString), createdAt: .now, kind: .subagent, parentSessionID: parent.id, rootSessionID:
↳ parent.rootSessionID, spawnedByRunID: parentRunID, spawnedByToolCallID: toolCallID, title: title, projectID: persistence.projectID,
↳ cwdRootBindingID: root.id)
        let runID = AgentRunID(UUID().uuidString)
        run = AgentRunInfo(runID: runID, sessionID: child.id, projectID: child.projectID, parentRunID: parentRunID, rootRunID: parentRun
↳ .rootRunID, agentKind: .subagent, status: .starting, modelSelection: resolved.selection, startedAt: .now, latestActivityAt: .now,
↳ title: title)
        try await persistence.createChildSessionAndRun(child, run: run, profile: normalizedProfile)
        runs[runID] = run
        runDeadlines[runID] = deadlinePolicy.deadline(for: .subagent, requested: normalizedProfile?.timeoutSeconds.map { .seconds($0) },
↳ parent: runDeadlines[parentRunID])
        if let normalizedProfile { executionProfiles[runID] = normalizedProfile }
    } else {
        child = try await store.create(kind: .subagent, parentSessionID: parent.id, rootSessionID: parent.rootSessionID, spawnedByRunID:
↳ parentRunID, spawnedByToolCallID: toolCallID, title: title)
        run = try await createRun(session: child, parentRunID: parentRunID, requestedModel: nil, resolvedModel: resolved, title: title,
↳ profile: normalizedProfile, emitEvent: false)
    }
    do {

```

```

        let runID = run.runID
        let status = await scheduler.submit(runID: runID) { [weak self] in await self?.runChild(runID: runID, task: task) }
        if status == .queued {
            run = updated(run, status: .queued)
            runs[run.runID] = run
            try await persistence?.saveAgentRun(run, profile: normalizedProfile)
        }
        await eventSink(.childSessionCreated(child.toInfo()))
        await eventSink(.agentRunStarted(run))
        await eventSink(.subagentSpawned(run))
        if status == .queued { await eventSink(.agentRunQueued(run)) }
        return (child.id, run)
    } catch {
        logDiagnostic("spawn failed parentSession=\\(parentSessionID.rawValue) parentRun=\\(parentRunID.rawValue) childSession=\\(child.id.
↳ rawValue): \\(error)")
        let childRuns = runs.values.filter { $0.sessionID == child.id }
        for r in childRuns {
            runs.removeValue(forKey: r.runID)
            runDeadlines.removeValue(forKey: r.runID)
            executionProfiles.removeValue(forKey: r.runID)
            try? await persistence?.deleteAgentRun(r.runID)
        }
        try? await store.deleteSession(child.id)
        throw error
    }
}

public func cancelAgentRun(_ runID: AgentRunID, descendants: Bool = true) async throws {
    _ = try agentRun(runID)
    let targets = descendants ? runs.values.filter { isDescendant($0, of: runID) || $0.runID == runID }.map(\\.runID) : [runID]
    for id in targets {
        await scheduler.cancel(id)
        let cancelled = await ProviderActivityRegistry.shared.cancelRun(id)
        for snapshot in cancelled {
            await eventSink(.providerActivityChanged(snapshot))
        }
        if let runtime = runtimes[runs[id]?.sessionID ?? SessionID("")] { await runtime.shutdown() }
        await finishRun(id, status: .cancelled, text: nil, usage: nil, error: CoreError(code: .toolCancelled, message: "AgentRun 已取消
↳ "))
    }
}

public func cancelSession(_ sessionID: SessionID) async {
    activeSessions.remove(sessionID)
    if let runtime = runtimes[sessionID] {
        await runtime.cancelCurrentTurn()
    }
}

/// 撤回时彻底重置并释放该 Session 的运行实例，消除脏缓存与残留任务，并递归取消所有子 Subagent
public func resetSessionForRevert(_ sessionID: SessionID) async {
    await cancelDescendants(parentSessionID: sessionID)
    activeSessions.remove(sessionID)
    if let runtime = runtimes.removeValue(forKey: sessionID) {
        await runtime.shutdown()
    }
}

public func cancelDescendants(parentSessionID: SessionID) async {
    let matchingRuns = runs.values.filter { run in
        run.sessionID == parentSessionID
    }
    for run in matchingRuns {
        try? await cancelAgentRun(run.runID, descendants: true)
    }
}

```

```

    if let allSessions = try? await store.listSessions() {
        let childSessions = allSessions.filter { $0.parentSessionID == parentSessionID || $0.rootSessionID == parentSessionID }
        for child in childSessions {
            activeSessions.remove(child.id)
            if let runtime = runtimes.removeValue(forKey: child.id) {
                await runtime.shutdown()
            }
        }
    }
}

public func cancelAgentRun(_ runID: AgentRunID, requester: AgentRunID) async throws {
    let run = try agentRun(runID)
    try requireSameTree(run, requester: requester)
    try await cancelAgentRun(runID)
}

public func continueChild(sessionID: SessionID, parentRunID: AgentRunID, content: String) async throws -> AgentRunInfo {
    let session = try await store.session(sessionID)
    guard session.kind == .subagent else { throw CoreError(code: .toolArgumentInvalid, message: "只能继续 Child Session") }
    let parentRun = try agentRun(parentRunID)
    guard let spawnedBy = session.spawnedByRunID, let original = runs[spawnedBy], original.rootRunID == parentRun.rootRunID else { throw
↳ CoreError(code: .permissionDenied, message: "Child Session 不属于当前 Agent 树") }
    guard !activeSessions.contains(sessionID) else { throw CoreError(code: .turnAlreadyRunning, message: "该 Child Session 已有进行中的
↳ AgentRun") }
    activeSessions.insert(sessionID)
    var run: AgentRunInfo
    do {
        run = try await createRun(session: session, parentRunID: parentRunID, requestedModel: nil, title: session.title)
    } catch {
        activeSessions.remove(sessionID)
        throw error
    }
    let runID = run.runID
    let status = await scheduler.submit(runID: runID) { [weak self] in await self?.runChild(runID: runID, task: content) }
    if status == .queued { run = updated(run, status: .queued); runs[runID] = run; try await persistence?.saveAgentRun(run) }
    if status == .queued { await eventSink(.agentRunQueued(run)) }
    await diagnostics?.record(kind: .subagent, event: status == .queued ? "agentRun.queued" : "agentRun.started", sessionID: run.
↳ sessionID, runID: run.runID, rootRunID: run.rootRunID, parentRunID: run.parentRunID, metadata: ["agentKind": run.agentKind.rawValue
↳ ])
    return run
}

public func markWaitingForQuestion(_ request: QuestionRequest, waiting: Bool) async {
    guard let runID = request.originRunID, let run = runs[runID], !run.status.isTerminal else { return }
    let updated = updated(run, status: waiting ? .waitingForUser : .running)
    runs[runID] = updated
    try? await persistence?.saveAgentRun(updated, profile: executionProfiles[runID])
    await eventSink(.agentRunStatusChanged(updated))
}

// MARK: - Private

private func makeRuntime(for sessionID: SessionID, run: AgentRunInfo? = nil, modelBus overrideBus: ModelBus? = nil, rootSessionID:
↳ SessionID? = nil) -> SessionRuntime {
    SessionRuntime(
        store: store,
        sessionID: sessionID,
        modelBus: overrideBus ?? modelBus,
        dataPlane: dataPlane,
        contextEngine: contextEngine,
        toolRuntime: toolRuntime,
        questions: questions,
        permissions: permissions,
        performanceStore: performanceStore,

```

```

    contextPager: contextPager,
    projectScanner: projectScanner,
    eventSink: eventSink,
    compactor: compactor,
    budgetPlanner: budgetPlanner,
    persistence: persistence,
    cacheController: cacheController,
    interactive: interactive,
    diagnosticsEnabled: diagnosticsEnabled,
    runID: run?.runID,
    rootRunID: run?.rootRunID,
    parentRunID: run?.parentRunID,
    rootSessionID: rootSessionID ?? sessionID,
    parentSessionID: run?.parentRunID.flatMap { runs[$0]?.sessionID },
    executionProfile: run.flatMap { executionProfiles[$0.runID] },
    systemContext: {
        let profile = run.flatMap { executionProfiles[$0.runID]?.behaviorProfile } ?? .build
        let exec = run.flatMap { executionProfiles[$0.runID] }
        let isFullAccess = exec?.permissionProfile == "fullAccess"
        return (behaviorInstructionsEnabled || profile != .build || isFullAccess) ? behaviorSystemContext(profile, exec) :
↪ behaviorSystemContext(.build, nil)
    }(),
    systemContextAtBeginning: behaviorInstructionsEnabled || run.flatMap { executionProfiles[$0.runID]?.behaviorProfile }.map { $0
↪ != .build } ?? false,
    maxAgentLoopSteps: maxAgentLoopSteps,
    runObserver: { [weak self] status, text, usage, error, terminalTrace in
        guard let run else { return }
        await self?.finishRun(run.runID, status: status, text: text, usage: usage, error: error, terminalTrace: terminalTrace)
    },
    deadlinePolicy: deadlinePolicy,
    restoreScheduler: restoreScheduler,
    diagnostics: diagnostics
)
}

private func runtime(for sessionID: SessionID, run: AgentRunInfo? = nil) async throws -> SessionRuntime {
    guard !shuttingDown else { throw CoreError(code: .notReady, message: "Agent 已关闭") }
    if let run {
        let resolved = try await modelResolver.resolve(run.modelSelection, subagent: run.agentKind == .subagent)
        let bus = ModelBus(gateway: ModelGateway(assembly: resolved.assembly, reasoning: run.modelSelection.reasoning, deadlinePolicy:
↪ deadlinePolicy))
        let session = try await store.session(sessionID)
        let runtime = makeRuntime(for: sessionID, run: run, modelBus: bus, rootSessionID: session.rootSessionID)
        runtimes[sessionID] = runtime
        try await runtime.restore()
        return runtime
    }
    if let runtime = runtimes[sessionID] { return runtime }
    _ = try await store.session(sessionID)
    let runtime = makeRuntime(for: sessionID)
    runtimes[sessionID] = runtime
    try await runtime.restore()
    return runtime
}

private func createRun(session: Session, parentRunID: AgentRunID?, requestedModel: ModelSelection? = nil, resolvedModel: (selection:
↪ ModelSelection, assembly: ModelRuntimeAssembly)? = nil, title: String?, profile: SubagentExecutionProfile? = nil, emitEvent: Bool =
↪ true) async throws -> AgentRunInfo {
    let effectiveProfile = profile ?? (session.kind == .primary ? behaviorProfile.executionProfile : nil)
    let resolved = try await (resolvedModel != nil ? resolvedModel! : modelResolver.resolve(requestedModel, subagent: session.kind == .
↪ subagent))
    let id = AgentRunID(UUID().uuidString)
    let root = parentRunID.flatMap { runs[$0]?.rootRunID } ?? id
    var selection = resolved.selection
    if selection.reasoning == nil, session.reasoningEffort != .auto {

```

```

        selection = ModelSelection(
            providerID: selection.providerID,
            accountID: selection.accountID,
            profileID: selection.profileID,
            modelID: selection.modelID,
            reasoning: session.reasoningEffort.rawValue,
            contextProfile: selection.contextProfile
        )
    }

    let run = AgentRunInfo(runID: id, sessionID: session.id, projectID: session.projectID, parentRunID: parentRunID, rootRunID: root,
    ↪ agentKind: session.kind, status: .starting, modelSelection: selection, startedAt: .now, latestActivityAt: .now, title: title)
    runs[id] = run
    runDeadlines[id] = deadlinePolicy.deadline(for: session.kind == .subagent ? .subagent : .agentRun, requested: effectiveProfile?.
    ↪ timeoutSeconds.map { .seconds($0) }, parent: parentRunID.flatMap { runDeadlines[$0] })
    if let effectiveProfile { executionProfiles[id] = effectiveProfile }
    do {
        try await persistence?.saveAgentRun(run, profile: effectiveProfile)
    } catch {
        runs.removeValue(forKey: id)
        runDeadlines.removeValue(forKey: id)
        executionProfiles.removeValue(forKey: id)
        throw error
    }
    if emitEvent {
        await eventSink(.agentRunStarted(run))
    }
    return run
}

private func runChild(runID: AgentRunID, task: String) async {
    guard let run = runs[runID] else { return }
    do {
        try await consumeChildTurn(run: run, task: task)
    } catch let error as CoreError {
        await finishRun(runID, status: [.commandTimedOut, .idleTimedOut].contains(error.code) ? .timedOut : .failed, text: nil, usage:
    ↪ nil, error: error)
    } catch {
        await finishRun(runID, status: .failed, text: nil, usage: nil, error: CoreError(code: .provider, message: String(describing:
    ↪ error)))
    }
}

private func consumeChildTurn(run: AgentRunInfo, task: String) async throws {
    let stream = try await runtime(for: run.sessionID, run: run).startTurn(task)
    for try await _ in stream.chunks {}
}

private func finishRun(_ runID: AgentRunID, status: AgentRunStatus, text: String?, usage: ModelUsage?, error: CoreError?, terminalTrace:
    ↪ AgentTerminalTrace? = nil) async {
    guard let old = runs[runID], !old.status.isTerminal else { return }
    if status.isTerminal {
        await toolRuntime.resetRun(runID)
    }
    let usage = AgentRunUsage(model: usage, elapsedMilliseconds: old.startedAt.map { Date().timeIntervalSince($0) * 1_000 })
    let trace = terminalTrace ?? AgentTerminalTrace(
        runID: old.runID,
        sessionID: old.sessionID,
        lastProviderRequestID: nil,
        finishReason: nil,
        lastToolCallID: nil,
        terminalTransition: "\\(old.status.rawValue) -> \\(status.rawValue)",
        terminalReason: status == .completed ? .completed : status == .cancelled ? .userCancelled : status == .timedOut ? .
    ↪ deadlineExceeded : .runtimeFailure,
        transitionSource: "AgentRuntime.finishRun",
        explanation: error?.message ?? (status == .completed ? "运行完成" : "运行终止")
    )
}

```

```

)
let run = AgentRunInfo(
  runID: old.runID,
  sessionID: old.sessionID,
  projectID: old.projectID,
  parentRunID: old.parentRunID,
  rootRunID: old.rootRunID,
  agentKind: old.agentKind,
  status: status,
  modelSelection: old.modelSelection,
  startedAt: old.startedAt,
  finishedAt: status.isTerminal ? .now : nil,
  latestActivityAt: .now,
  error: error,
  usage: usage,
  title: old.title,
  terminalReason: status.isTerminal ? trace.terminalReason : nil,
  terminalTrace: status.isTerminal ? trace : nil
)
let result = status.isTerminal ? SubagentResult(
  childSessionID: run.sessionID,
  runID: runID,
  status: status,
  finalText: text,
  usage: usage,
  error: error,
  terminalReason: trace.terminalReason,
  terminalTrace: trace
) : nil
do {
  if let result {
    try await persistence?.saveTerminalAgentRun(run, result: result, profile: executionProfiles[runID])
  } else {
    try await persistence?.saveAgentRun(run, profile: executionProfiles[runID])
  }
} catch {
  let persistenceError = CoreError(code: .persistence, message: "AgentRun 持久化失败: \(String(describing: error))")
  let failed = AgentRunInfo(runID: old.runID, sessionID: old.sessionID, projectID: old.projectID, parentRunID: old.parentRunID,
↪ rootRunID: old.rootRunID, agentKind: old.agentKind, status: .failed, modelSelection: old.modelSelection, startedAt: old.startedAt,
↪ finishedAt: .now, latestActivityAt: .now, error: persistenceError, usage: usage, title: old.title)
  runs[runID] = failed
  activeSessions.remove(failed.sessionID)
  let failedResult = SubagentResult(childSessionID: failed.sessionID, runID: runID, status: .failed, usage: usage, error:
↪ persistenceError)
  results[runID] = failedResult
  resumeWaiters(for: runID, result: failedResult)
  await scheduler.complete(runID)
  await eventSink(.agentRunFailed(failed))
  await eventSink(.subagentResultAvailable(failedResult))
  logDiagnostic("finishRun persistence failed run=\(runID.rawValue): \(error)")
  return
}
runs[runID] = run
Task { await diagnostics?.record(kind: status == .completed ? .agentRun : .error, event: "agentRun.\(status.rawValue)", sessionID:
↪ run.sessionID, runID: run.runID, rootRunID: run.rootRunID, parentRunID: run.parentRunID, errorCode: error?.code.rawValue) }
if status.isTerminal { activeSessions.remove(run.sessionID) }
if status.isTerminal { runDeadlines.removeValue(forKey: runID) }
if let result {
  results[runID] = result
  resumeWaiters(for: runID, result: result)
  await scheduler.complete(runID)
  await eventSink(status == .completed ? .agentRunCompleted(run) : status == .cancelled ? .agentRunCancelled(run) : .
↪ agentRunFailed(run))
  await eventSink(.subagentResultAvailable(result))
} else {

```

```

        await eventSink(.agentRunStatusChanged(run))
    }
}

private func depth(of session: Session) async throws -> Int {
    var result = 0; var current = session.parentSessionID
    while let id = current { result += 1; current = try await store.session(id).parentSessionID }
    return result
}

private func isDescendant(_ run: AgentRunInfo, of ancestor: AgentRunID) -> Bool {
    var current = run.parentRunID
    while let id = current { if id == ancestor { return true }; current = runs[id]?.parentRunID }
    return false
}

private func requireSameTree(_ target: AgentRunInfo, requester: AgentRunID) throws {
    guard let requesterRun = runs[requester], requesterRun.rootRunID == target.rootRunID else {
        throw CoreError(code: .permissionDenied, message: "AgentRun 不属于当前 Agent 树")
    }
}

private func resumeWaiters(for runID: AgentRunID, result: SubagentResult) {
    for waiter in resultWaiters.removeValue(forKey: runID) ?? [] { waiter.resume(returning: result) }
}

private func updated(_ run: AgentRunInfo, status: AgentRunStatus) -> AgentRunInfo {
    AgentRunInfo(runID: run.runID, sessionID: run.sessionID, projectID: run.projectID, parentRunID: run.parentRunID, rootRunID: run.
↳ rootRunID, agentKind: run.agentKind, status: status, modelSelection: run.modelSelection, startedAt: run.startedAt, finishedAt: run.
↳ finishedAt, latestActivityAt: .now, error: run.error, usage: run.usage, title: run.title)
}

private func normalized(profile: SubagentExecutionProfile?, parent: SubagentExecutionProfile?) throws -> SubagentExecutionProfile? {
    guard let profile else { return parent }
    let rawPermission: String?
    if let raw = profile.permissionProfile {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty, ExecutionProfile(rawValue: trimmed) != nil else {
            throw CoreError(code: .toolArgumentInvalid, message: "未知 permissionProfile: \"(raw)\"")
        }
        rawPermission = trimmed
    } else {
        rawPermission = nil
    }

    let budget: String?
    if let raw = profile.budgetProfile {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty, let val = Int(trimmed), val > 0 else {
            throw CoreError(code: .toolArgumentInvalid, message: "budgetProfile 必须是正整数 token budget")
        }
        budget = trimmed
    } else {
        budget = nil
    }

    let context: String?
    if let raw = profile.contextProfile {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty, let val = Int(trimmed), val > 0 else {
            throw CoreError(code: .toolArgumentInvalid, message: "contextProfile 必须是正整数 context window")
        }
        context = trimmed
    } else {
        context = nil
    }
}

```

```

    }

    if let value = profile.maxSteps, value <= 0 { throw CoreError(code: .toolArgumentInvalid, message: "maxSteps 必须大于 0") }
    if let value = profile.timeoutSeconds, value <= 0 { throw CoreError(code: .toolArgumentInvalid, message: "timeoutSeconds 必须大于
    ↪ 0") }
    let childTools = profile.toolProfile.map(Set.init)
    let parentTools = parent?.toolProfile.map(Set.init)
    let tools: [String]? = switch (childTools, parentTools) {
    case let (.some(child), .some(parent)): Array(child.intersection(parent)).sorted()
    case let (.some(child), .none): Array(child).sorted()
    case let (.none, .some(parent)): Array(parent).sorted()
    case (.none, .none): nil
    }
    let rank: [ExecutionProfile: Int] = [.readOnly: 0, .workspace: 1, .fullAccess: 2]
    let requestedPermission = rawPermission.flatMap(ExecutionProfile.init(rawValue:))
    let parentPermission = parent?.permissionProfile.flatMap(ExecutionProfile.init(rawValue:))
    let permission: ExecutionProfile? = switch (requestedPermission, parentPermission) {
    case let (.some(child), .some(parent)): rank[child, default: 0] <= rank[parent, default: 0] ? child : parent
    case let (.some(child), .none): child
    case let (.none, .some(parent)): parent
    case (.none, .none): nil
    }
    return SubagentExecutionProfile(modelSelection: profile.modelSelection, permissionProfile: permission?.rawValue, toolProfile: tools,
    ↪ budgetProfile: budget ?? parent?.budgetProfile, contextProfile: context ?? parent?.contextProfile, maxSteps: profile.maxSteps,
    ↪ timeoutSeconds: profile.timeoutSeconds)
}

private func logDiagnostic(_ message: String) {
    guard diagnosticsEnabled else { return }
}
}

```

// File: Sources/LingXiCore/Modules/Agent/WorkflowRuntime.swift

```

import Foundation
import LingXiProtocol

public struct WorkflowTaskCompletion: Sendable, Equatable {
    public let result: SubagentResult?
    public let error: CoreError?

    public init(result: SubagentResult? = nil, error: CoreError? = nil) {
        self.result = result
        self.error = error
    }
}

/// Durable DAG coordinator. It never replays a task after restart: interrupted work is recoveryRequired.
public actor WorkflowRuntime {
    public typealias Executor = @Sendable (WorkflowID, WorkflowTaskDefinition, WorkflowTaskProvenance, @escaping @Sendable (
    ↪ WorkflowTaskProvenance) async -> Void) async -> WorkflowTaskCompletion
    public typealias InputSink = @Sendable (WorkflowID, WorkflowTaskID, WorkflowPendingInput) async -> Void

    private let persistence: SQLitePersistenceStore?
    private let executor: Executor?
    private let diagnostics: RuntimeDiagnosticsStore?
    private var inputSink: InputSink?
    private var workflows: [WorkflowID: WorkflowSnapshot] = [:]
    private var active: [String: Task<Void, Never>] = [:]

    public init(persistence: SQLitePersistenceStore? = nil, executor: Executor? = nil, diagnostics: RuntimeDiagnosticsStore? = nil) {
        self.persistence = persistence
        self.executor = executor
        self.diagnostics = diagnostics
    }
}

```

```

public func setInputSink(_ sink: @escaping InputSink) {
    inputSink = sink
}

public func restore() async throws {
    for stored in try await persistence?.loadWorkflows() ?? [] {
        let recovered = recover(stored)
        workflows[recovered.id] = recovered
        if recovered != stored { try await persist(recovered) }
    }
}

public func create(id: WorkflowID = WorkflowID(UUID().uuidString), rootSessionID: SessionID, rootRunID: AgentRunID, tasks: [
    ↪ WorkflowTaskDefinition]) async throws -> WorkflowSnapshot {
    try validate(tasks)
    let snapshot = WorkflowSnapshot(id: id, rootSessionID: rootSessionID, rootRunID: rootRunID, tasks: tasks.map { WorkflowTaskState(
    ↪ definition: $0) })
    workflows[id] = snapshot
    await diagnostics?.record(kind: .workflow, event: "workflow.created", sessionID: rootSessionID, runID: rootRunID, workflowID: id,
    ↪ metadata: ["taskCount": String(tasks.count)])
    try await persist(snapshot)
    await schedule(id)
    return workflows[id]!
}

public func workflow(_ id: WorkflowID) throws -> WorkflowSnapshot {
    guard let workflow = workflows[id] else { throw CoreError(code: .resourceNotFound, message: "Workflow 不存在: \(id.rawValue)") }
    return workflow
}

public func allWorkflows() -> [WorkflowSnapshot] { workflows.values.sorted { $0.createdAt < $1.createdAt } }

public func readyTasks(_ id: WorkflowID) throws -> [WorkflowTaskID] {
    let workflow = try workflow(id)
    return workflow.tasks.filter { $0.status == .pending && dependenciesCompleted($0, in: workflow) }.map(\.definition.id)
}

/// Stores the origin-owned input request. The actual continuation remains in the originating runtime only.
public func suspend(workflowID: WorkflowID, taskID: WorkflowTaskID, input: WorkflowPendingInput) async throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }) else { throw CoreError(code: .resourceNotFound,
    ↪ message: "Workflow Task 不存在") }
    let status: WorkflowTaskStatus
    switch input {
    case .question: status = .waitingForQuestion
    case .permission: status = .waitingForPermission
    case .decision: status = .waitingForDecision
    }
    workflow = replacing(workflow, task: replace(workflow.tasks[index], status: status, pendingInput: input), status: .waitingForUser,
    ↪ checkpoint: checkpoint(workflow, taskID, "input-pending"))
    workflows[workflowID] = workflow
    await diagnostics?.record(kind: .hitl, event: "workflow.input.pending", sessionID: workflow.rootSessionID, runID: workflow.rootRunID
    ↪ , workflowID: workflowID, taskID: taskID)
    try await persist(workflow)
    if case .decision = input {
        await inputSink?(workflowID, taskID, input)
    }
}

/// Records that routing has been re-established. It does not rerun or complete the originating task.
public func reattachPendingInput(workflowID: WorkflowID, taskID: WorkflowTaskID) async throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }), workflow.tasks[index].pendingInput != nil else {
    ↪ throw CoreError(code: .resourceNotFound, message: "Workflow Task 没有挂起输入") }

```

```

        workflow = replacing(workflow, task: replace(workflow.tasks[index], status: workflow.tasks[index].status), status: .waitingForUser,
        ↪ checkpoint: checkpoint(workflow, taskID, "input-reattached"))
        workflows[workflowID] = workflow
        await diagnostics?.record(kind: .recovery, event: "workflow.recovery.acknowledged", sessionID: workflow.rootSessionID, runID:
        ↪ workflow.rootRunID, workflowID: workflowID, taskID: taskID)
        try await persist(workflow)
    }

public func suspendForOrigin(runID: AgentRunID, input: WorkflowPendingInput) async throws {
    guard let match = workflows.values.lazy.flatMap(\.tasks).first(where: { $0.provenance?.childRunID == runID }), let workflow =
    ↪ workflows.values.first(where: { $0.tasks.contains(where: { $0.definition.id == match.definition.id && $0.provenance?.childRunID ==
    ↪ runID }) }) else {
        throw CoreError(code: .resourceNotFound, message: "Workflow child Run 不存在")
    }
    try await suspend(workflowID: workflow.id, taskID: match.definition.id, input: input)
}

public func suspendForOrigin(sessionID: SessionID, input: WorkflowPendingInput) async throws {
    guard let match = workflows.values.lazy.flatMap(\.tasks).first(where: { $0.provenance?.childSessionID == sessionID }), let workflow
    ↪ = workflows.values.first(where: { $0.tasks.contains(where: { $0.definition.id == match.definition.id && $0.provenance?.
    ↪ childSessionID == sessionID }) }) else {
        throw CoreError(code: .resourceNotFound, message: "Workflow child Session 不存在")
    }
    try await suspend(workflowID: workflow.id, taskID: match.definition.id, input: input)
}

public func complete(workflowID: WorkflowID, taskID: WorkflowTaskID, result: SubagentResult? = nil, error: CoreError? = nil) async
    ↪ throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }) else { throw CoreError(code: .resourceNotFound,
    ↪ message: "Workflow Task 不存在") }
    let current = workflow.tasks[index]
    guard !current.status.isTerminal else { return }
    let status: WorkflowTaskStatus = error == nil ? .completed : (error?.code == .commandTimedOut ? .timedOut : .failed)
    let provenance = result.map { WorkflowTaskProvenance(parentSessionID: workflow.rootSessionID, parentRunID: workflow.rootRunID,
    ↪ childSessionID: $0.childSessionID, childRunID: $0.runID) }
    workflow = replacing(workflow, task: replace(current, status: status, provenance: provenance, result: result, pendingInput: nil,
    ↪ error: error), status: status == .completed ? .running : .failed, checkpoint: checkpoint(workflow, taskID, "task-\(status.rawValue)
    ↪ "))
    workflow = blockDependents(in: workflow)
    workflow = withDerivedStatus(workflow)
    workflows[workflowID] = workflow
    active.removeValue(forKey: key(workflowID, taskID))
    try await persist(workflow)
    await schedule(workflowID)
}

/// An operator may explicitly make an interrupted task eligible again after verifying its side effects.
public func acknowledgeRecovery(workflowID: WorkflowID, taskID: WorkflowTaskID) async throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }), workflow.tasks[index].status == .
    ↪ recoveryRequired else { throw CoreError(code: .toolArgumentInvalid, message: "Task 不处于 recoveryRequired") }
    workflow = replacing(workflow, task: replace(workflow.tasks[index], status: .pending), status: .running, checkpoint: checkpoint(
    ↪ workflow, taskID, "recovery-acknowledged"))
    workflows[workflowID] = workflow
    try await persist(workflow)
    await schedule(workflowID)
}

private func schedule(_ workflowID: WorkflowID) async {
    guard var workflow = workflows[workflowID], workflow.status == .pending || workflow.status == .running else { return }
    var changed = false
    for index in workflow.tasks.indices where workflow.tasks[index].status == .pending && dependenciesCompleted(workflow.tasks[index],
    ↪ in: workflow) {
        guard let executor, workflow.tasks[index].definition.kind == .agent else { continue }

```

```

        let task = workflow.tasks[index]
        let provenance = WorkflowTaskProvenance(parentSessionID: workflow.rootSessionID, parentRunID: workflow.rootRunID)
        workflow = replacing(workflow, task: replace(task, status: .running, provenance: provenance), status: .running, checkpoint:
↳ checkpoint(workflow, task.definition.id, "task-running"))
        changed = true
        let workflowID = workflow.id
        let taskID = task.definition.id
        active[key(workflowID, taskID)] = Task.detached { [weak self] in
            let completion = await executor(workflowID, task.definition, provenance) { [weak self] child in
                await self?.recordChild(workflowID: workflowID, taskID: taskID, provenance: child)
            }
            try? await self?.complete(workflowID: workflowID, taskID: taskID, result: completion.result, error: completion.error)
        }
    }
    if changed {
        workflows[workflowID] = workflow
        try? await persist(workflow)
    }
}

private func recover(_ workflow: WorkflowSnapshot) -> WorkflowSnapshot {
    let tasks = workflow.tasks.map { task in
        task.status == .running ? replace(task, status: .recoveryRequired) : task
    }
    let status: WorkflowStatus = tasks.contains(where: { $0.status == .recoveryRequired }) ? .recoveryRequired : workflow.status
    return WorkflowSnapshot(id: workflow.id, rootSessionID: workflow.rootSessionID, rootRunID: workflow.rootRunID, status: status, tasks
↳ : tasks, checkpoint: WorkflowCheckpoint(sequence: workflow.checkpoint.sequence + 1, label: "restart-recovered"), createdAt:
↳ workflow.createdAt, updatedAt: .now)
}

private func recordChild(workflowID: WorkflowID, taskID: WorkflowTaskID, provenance: WorkflowTaskProvenance) async {
    guard var workflow = workflows[workflowID], let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }), workflow.
↳ tasks[index].status == .running else { return }
    workflow = replacing(workflow, task: replace(workflow.tasks[index], status: .running, provenance: provenance), status: .running,
↳ checkpoint: checkpoint(workflow, taskID, "child-started"))
    workflows[workflowID] = workflow
    try? await persist(workflow)
}

private func validate(_ tasks: [WorkflowTaskDefinition]) throws {
    guard !tasks.isEmpty, Set(tasks.map(\.id)).count == tasks.count else { throw CoreError(code: .toolArgumentInvalid, message: "
↳ Workflow Task ID 必须唯一且非空") }
    let ids = Set(tasks.map(\.id))
    guard tasks.allSatisfy({ !$0.dependencies.contains($0.id) && Set($0.dependencies).count == $0.dependencies.count && Set($0.
↳ dependencies).isSubset(of: ids) }) else { throw CoreError(code: .toolArgumentInvalid, message: "Workflow dependency 无效") }
    var remaining = Set(tasks.map(\.id))
    while let next = tasks.first(where: { remaining.contains($0.id) && $0.dependencies.allSatisfy { !remaining.contains($0) } }) {
        remaining.remove(next.id)
    }
    guard remaining.isEmpty else { throw CoreError(code: .toolArgumentInvalid, message: "Workflow dependency 不能成环") }
}

private func dependenciesCompleted(_ task: WorkflowTaskState, in workflow: WorkflowSnapshot) -> Bool {
    task.definition.dependencies.allSatisfy { id in workflow.tasks.first(where: { $0.definition.id == id })?.status == .completed }
}

private func blockDependents(in workflow: WorkflowSnapshot) -> WorkflowSnapshot {
    var result = workflow
    for task in workflow.tasks where task.status == .pending && task.definition.dependencies.contains(where: { id in
        guard let status = result.tasks.first(where: { $0.definition.id == id })?.status else { return true }
        return status == .failed || status == .cancelled || status == .timedOut || status == .blocked
    }) {
        result = replacing(result, task: replace(task, status: .blocked, error: CoreError(code: .toolExecutionFailed, message: "Workflow
↳ dependency 未成功完成")), status: result.status, checkpoint: result.checkpoint)
    }
}

```

```

        return result
    }

    private func withDerivedStatus(_ workflow: WorkflowSnapshot) -> WorkflowSnapshot {
        let status: WorkflowStatus
        if workflow.tasks.allSatisfy({ $0.status == .completed }) { status = .completed }
        else if workflow.tasks.contains(where: { $0.status == .waitingForQuestion || $0.status == .waitingForPermission || $0.status == .
        ↪ waitingForDecision }) { status = .waitingForUser }
        else if workflow.tasks.contains(where: { $0.status == .recoveryRequired }) { status = .recoveryRequired }
        else if workflow.tasks.contains(where: { $0.status == .failed || $0.status == .timedOut || $0.status == .blocked }) { status = .
        ↪ failed }
        else { status = .running }
        return WorkflowSnapshot(id: workflow.id, rootSessionID: workflow.rootSessionID, rootRunID: workflow.rootRunID, status: status, tasks
        ↪ : workflow.tasks, checkpoint: workflow.checkpoint, createdAt: workflow.createdAt, updatedAt: .now)
    }

    private func replacing(_ workflow: WorkflowSnapshot, task: WorkflowTaskState, status: WorkflowStatus, checkpoint: WorkflowCheckpoint) ->
    ↪ WorkflowSnapshot {
        WorkflowSnapshot(id: workflow.id, rootSessionID: workflow.rootSessionID, rootRunID: workflow.rootRunID, status: status, tasks:
        ↪ workflow.tasks.map { $0.definition.id == task.definition.id ? task : $0 }, checkpoint: checkpoint, createdAt: workflow.createdAt,
        ↪ updatedAt: .now)
    }

    private func replace(_ task: WorkflowTaskState, status: WorkflowTaskStatus, provenance: WorkflowTaskProvenance? = nil, result:
    ↪ SubagentResult? = nil, pendingInput: WorkflowPendingInput? = nil, error: CoreError? = nil) -> WorkflowTaskState {
        WorkflowTaskState(definition: task.definition, status: status, provenance: provenance ?? task.provenance, result: result ?? task.
        ↪ result, pendingInput: pendingInput, error: error ?? task.error)
    }

    private func checkpoint(_ workflow: WorkflowSnapshot, _ taskID: WorkflowTaskID?, _ label: String) -> WorkflowCheckpoint {
        WorkflowCheckpoint(sequence: workflow.checkpoint.sequence + 1, taskID: taskID, label: label)
    }

    private func persist(_ workflow: WorkflowSnapshot) async throws { try await persistence?.saveWorkflow(workflow) }
    private func key(_ workflowID: WorkflowID, _ taskID: WorkflowTaskID) -> String { "\(workflowID.rawValue)/\(taskID.rawValue)" }
}

```

// File: Sources/LingXiCore/Modules/Agent/AgentInstructions.swift

```

import Foundation
import LingXiProtocol

struct AgentInstruction: Sendable, Equatable {
    let source: String
    let scope: String
    let content: String
    let global: Bool
}

struct AgentInstructionSet: Sendable {
    private let workspace: URL
    private let instructions: [AgentInstruction]

    static func load(workspace: URL, globalInstructionsURL: URL? = defaultGlobalInstructionsURL) throws -> Self {
        let root = workspace.resolvingSymlinksInPath().standardizedFileURL
        let skipped = Set([".build", ".git", ".dev-sandbox", ".dev-sandbox-backups", "node_modules"])
        let enumerator = FileManager.default.enumerator(
            at: root,
            includingPropertiesForKeys: [.isDirectoryKey, .isRegularFileKey, .fileSizeKey],
            options: [.skipsPackageDescendants]
        )
        var instructions = (try globalInstructionsURL.flatMap(loadGlobal)).map { [$0] } ?? []

        while let candidate = enumerator?.nextObject() as? URL {
            let values = try candidate.resourceValues(forKeys: [.isDirectoryKey, .isRegularFileKey, .fileSizeKey])

```

```

    if values.isDirectory == true, skipped.contains(candidate.lastPathComponent) {
        enumerator?.skipDescendants()
        continue
    }
    guard candidate.lastPathComponent == "AGENTS.md", values.isRegularFile == true else { continue }
    let resolved = candidate.resolvingSymlinksInPath().standardizedFileURL
    guard resolved.lastPathComponent == "AGENTS.md", contains(root, resolved), (values.fileSize ?? 0) <= 64 * 1_024 else { continue
↵ }
    guard let content = try? String(contentsOf: resolved, encoding: .utf8) else {
        throw CoreError(code: .toolExecutionFailed, message: "无法读取 AGENTS.md: \"(relative(resolved, to: root))\"")
    }
    let trimmed = content.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !trimmed.isEmpty else { continue }
    instructions.append(AgentInstruction(
        source: relative(resolved, to: root),
        scope: relative(resolved.deletingLastPathComponent(), to: root),
        content: trimmed,
        global: false
    ))
}
return Self(workspace: root, instructions: instructions.sorted {
    $0.global != $1.global ? $0.global : $0.scope.count < $1.scope.count
})
}

func applicable(to target: URL) -> [AgentInstruction] {
    let resolved = target.resolvingSymlinksInPath().standardizedFileURL
    guard Self.contains(workspace, resolved) else { return [] }
    return instructions.filter { instruction in
        if instruction.global { return true }
        let scope = instruction.scope == "." ? workspace : workspace.appendingPathComponent(instruction.scope).standardizedFileURL
        return Self.contains(scope, resolved)
    }
}

func rendered() -> String? {
    guard !instructions.isEmpty else { return nil }
    let entries = instructions.enumerated().map { offset, instruction in
        "[AGENTS source=\"(instruction.source)\" scope=\"(instruction.scope)\" priority=\"(offset + 1)\"\\n\\n(instruction.content)\"
    }
    return "Repository instructions are scoped by target path. Apply only sources whose scope contains the target; a higher priority (
↵ closer scope) overrides a conflicting lower priority instruction. Runtime safety and the active execution profile cannot be
↵ overridden.\\n\\n" + entries.joined(separator: "\\n\\n")
}

private static func contains(_ parent: URL, _ child: URL) -> Bool {
    child.path == parent.path || child.path.hasPrefix(parent.path + "/" )
}

private static var defaultGlobalInstructionsURL: URL {
    FileManager.default.homeDirectoryForCurrentUser.appendingPathComponent(".lingxiagent/AGENTS.md")
}

private static func loadGlobal(_ url: URL) throws -> AgentInstruction? {
    guard FileManager.default.fileExists(atPath: url.path) else { return nil }
    let resolved = url.resolvingSymlinksInPath().standardizedFileURL
    let values = try resolved.resourceValues(forKeys: [.isRegularFileKey, .fileSizeKey])
    guard resolved.lastPathComponent == "AGENTS.md", values.isRegularFile == true, (values.fileSize ?? 0) <= 64 * 1_024 else { return
↵ nil }
    guard let content = try? String(contentsOf: resolved, encoding: .utf8) else {
        throw CoreError(code: .toolExecutionFailed, message: "无法读取全局 AGENTS.md")
    }
    let trimmed = content.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !trimmed.isEmpty else { return nil }
    return AgentInstruction(source: "~/lingxiagent/AGENTS.md", scope: "global", content: trimmed, global: true)
}

```

```

    }

    private static func relative(_ url: URL, to root: URL) -> String {
        let path = url.path
        let prefix = root.path == "/" ? "/" : root.path + "/"
        return path.hasPrefix(prefix) ? String(path.dropFirst(prefix.count)) : "."
    }
}

public struct AgentEnvironmentFacts: Sendable, Equatable {
    public let platform: String
    public let workspaceRoot: String
    public let currentDirectory: String
    public let homeDirectory: String
    public let shell: String
    public let isGitRepository: Bool
    public let gitBranch: String?
    public let accessScope: String

    public init(
        platform: String = LingXiPlatform.system.osName,
        workspaceRoot: String,
        currentDirectory: String = FileManager.default.currentDirectoryPath,
        homeDirectory: String = FileManager.default.homeDirectoryForCurrentUser.path,
        shell: String = "unknown",
        isGitRepository: Bool? = nil,
        gitBranch: String? = nil,
        accessScope: String = "workspace"
    ) {
        self.platform = platform
        self.workspaceRoot = workspaceRoot
        self.currentDirectory = currentDirectory
        self.homeDirectory = homeDirectory
        self.shell = shell
        let gitRoot = URL(fileURLWithPath: workspaceRoot).appendingPathComponent(".git")
        self.isGitRepository = isGitRepository ?? FileManager.default.fileExists(atPath: gitRoot.path)
        if let gitBranch {
            self.gitBranch = gitBranch
        } else if let head = try? String(contentsOf: gitRoot.appendingPathComponent("HEAD"), encoding: .utf8), head.hasPrefix("ref: refs/heads/") {
            self.gitBranch = String(head.dropFirst("ref: refs/heads/".count)).trimmingCharacters(in: .whitespacesAndNewlines)
        } else {
            self.gitBranch = nil
        }
        self.accessScope = accessScope
    }
}

public func render() -> String {
    """
    Environment facts:
    - platform: \(platform)
    - workspaceRoot: \(workspaceRoot)
    - cwd: \(currentDirectory)
    - userHome: \(homeDirectory)
    - shell: \(shell)
    - gitRepository: \(isGitRepository ? "yes" : "no")\(gitBranch.map { " · branch: \($0)" } ?? "")
    - accessScope: \(accessScope)
    """
}

enum AgentBehaviorInstructions {
    static let runtimeGuidelines = """
    Agent Runtime Guidelines:
    - File Operations Protocol (CRITICAL):

```

- * Always use dedicated file mutation tools: `write\_file` (for creating new files or full overwrites) and `edit\_file` or `apply\_patch`
↳ (for surgical edits and patches).
- * NEVER use shell/bash commands (such as `cat <<EOF`, `echo >`, `sed`, `awk`, or redirection scripts) to write, create, or modify
↳ files. Shell execution is strictly reserved for compilation, testing, package management, git commands, and process execution.
- Task Planning: For multi-step tasks, investigations, or refactoring, proactively use `todo` (action: 'add') to establish a checklist,
↳ and update task status ('in_progress', 'completed', 'failed') as you advance to keep the sidebar updated.
- Parallel Tool Calling: When you need to read multiple files, inspect directories, grep across files, or perform independent read-only
↳ investigations, emit multiple tool calls in parallel within the same turn instead of waiting for sequential round-trips. The
↳ runtime executes independent tool calls concurrently.
- Tool Discovery: Builtin tools are always available for filesystem, grep, and execution. If `search\_tools` returns no matches or a
↳ diagnostic notice (empty/error), do not retry searching; proceed with builtin tools.
- Execution & Truthfulness: Inspect before mutating, run verification after changes, and report obstacles truthfully without
↳ hallucination.

```
""""
```

```
static func render(
    profile: AgentBehaviorProfile,
    configured: String?,
    repository: AgentInstructionSet,
    environmentFacts: AgentEnvironmentFacts? = nil
) -> String? {
    var entries: [String] = []
    if let facts = environmentFacts {
        entries.append(facts.render())
    }
    switch profile {
    case .build:
        entries.append("Build profile: inspect before editing; after every mutation, run the narrowest relevant verification. On tool  
↳ failure or timeout, use returned diagnostics to change strategy or report the blocker. Before completion, inspect the diff and  
↳ verification result. Do not repeat an identical failed action.")
    case .plan:
        entries.append("Plan profile: investigate with read-only tools and return an executable plan with evidence. Repository mutation  
↳ is forbidden by runtime capability policy.")
    case .explore:
        entries.append("Explore profile: use read-only search and inspection, report evidence and uncertainty, and do not mutate the  
↳ repository. Mutation is forbidden by runtime capability policy.")
    }
    entries.append(runtimeGuidelines)
    if let configured, !configured.isEmpty { entries.append(configured) }
    if let repository = repository.rendered() { entries.append(repository) }
    return entries.joined(separator: "\n\n")
}
}
```

// File: Sources/LingXiCore/Modules/Agent/AgentRunRuntime.swift

```
import Foundation
import LingXiProtocol

public struct SubagentRuntimeLimits: Sendable, Equatable {
    public let maxConcurrentSubagents: Int
    public let maxSubagentDepth: Int
    public let maxTotalRunsPerRootRun: Int

    public init(maxConcurrentSubagents: Int = 4, maxSubagentDepth: Int = 3, maxTotalRunsPerRootRun: Int = 32) {
        self.maxConcurrentSubagents = max(1, maxConcurrentSubagents)
        self.maxSubagentDepth = max(1, maxSubagentDepth)
        self.maxTotalRunsPerRootRun = max(1, maxTotalRunsPerRootRun)
    }
}

public actor SubagentModelResolver {
    private var runtimes: [String: ModelRuntimeAssembly]
    private let allowedModels: Set<String>?
    private var defaultSelection: ModelSelection?
```

```

private let defaultSubagentSelection: ModelSelection?

public init(defaultRuntime: ModelRuntimeAssembly, runtimes: [String: ModelRuntimeAssembly] = [:], allowedModels: Set<String>? = nil,
↳ defaultSelection: ModelSelection? = nil, defaultSubagentSelection: ModelSelection? = nil) {
    var values = runtimes
    values["default"] = values["default"] ?? defaultRuntime
    values[defaultRuntime.endpoint.providerID] = values[defaultRuntime.endpoint.providerID] ?? defaultRuntime
    self.runtimes = values
    self.allowedModels = allowedModels
    self.defaultSelection = defaultSelection
    self.defaultSubagentSelection = defaultSubagentSelection
}

public func registerAssembly(_ assembly: ModelRuntimeAssembly, for selection: ModelSelection) {
    let providerID = selection.providerID
    runtimes[providerID] = assembly
    runtimes["default"] = assembly
    if let accountID = selection.accountID, let profileID = selection.profileID {
        runtimes["\\(accountID)::\\(profileID)"] = assembly
    }
}

public func resolve(_ requested: ModelSelection?, subagent: Bool = false) throws -> (selection: ModelSelection, assembly:
↳ ModelRuntimeAssembly) {
    let requested = requested ?? (subagent ? defaultSubagentSelection ?? defaultSelection : defaultSelection)
    let providerID = requested?.providerID ?? "default"
    guard (requested?.accountID == nil) == (requested?.profileID == nil) else { throw CoreError(code: .subagentModelNotAllowed, message:
↳ "ModelSelection 必须同时提供 accountID 与 profileID") }
    let key = requested?.accountID.flatMap { account in requested?.profileID.map { "\\(account)::\\($0)" } }
    var assembly = key.flatMap({ runtimes[$0] }) ?? (requested?.accountID == nil ? runtimes[providerID] : nil)
    if assembly == nil {
        assembly = runtimes[providerID] ?? runtimes["default"]
    }
    guard let resolvedAssembly = assembly else { throw CoreError(code: .subagentModelNotAllowed, message: "Subagent Provider 不可用: \\(
↳ providerID)") }
    guard !resolvedAssembly.modelID.rawValue.isEmpty else { throw CoreError(code: .provider, message: "未配置模型 Provider") }

    let selection = requested ?? ModelSelection(providerID: providerID, modelID: resolvedAssembly.modelID.rawValue)
    guard allowedModels?.contains(selection.modelID) ?? true else { throw CoreError(code: .subagentModelNotAllowed, message: "Subagent
↳ Model 未获用户许可: \\(selection.modelID)") }

    // 若 modelID 属于该 provider 且与初始 assembly 声明的缺省 modelID 不同, 动态派生出对应 modelID 的 assembly
    let effectiveAssembly: ModelRuntimeAssembly
    if selection.modelID != resolvedAssembly.modelID.rawValue {
        effectiveAssembly = ModelRuntimeAssembly(
            provider: resolvedAssembly.provider,
            modelID: ModelID(selection.modelID),
            contextProfile: resolvedAssembly.contextProfile,
            endpoint: ResolvedModelEndpoint(
                providerID: resolvedAssembly.endpoint.providerID,
                productID: resolvedAssembly.endpoint.productID,
                endpointID: resolvedAssembly.endpoint.endpointID,
                accountID: resolvedAssembly.endpoint.accountID,
                profileID: selection.profileID ?? resolvedAssembly.endpoint.profileID,
                modelID: ModelID(selection.modelID),
                baseURL: resolvedAssembly.endpoint.baseURL,
                wireProtocol: resolvedAssembly.endpoint.wireProtocol,
                contextProfile: resolvedAssembly.endpoint.contextProfile,
                capabilities: resolvedAssembly.endpoint.capabilities,
                rateLimits: resolvedAssembly.endpoint.rateLimits
            )
        )
    } else {
        effectiveAssembly = resolvedAssembly
    }
}

```

```

        return (selection, effectiveAssembly)
    }

    public func setDefaultSelection(_ selection: ModelSelection, assembly: ModelRuntimeAssembly? = nil) throws {
        if let assembly {
            registerAssembly(assembly, for: selection)
        }
        _ = try resolve(selection)
        defaultSelection = selection
    }

    public func currentDefaultSelection() -> ModelSelection? { defaultSelection }
}

/// One scheduler owns queuing, limits, and cancellation for every descendant run.
public actor AgentRunScheduler {
    private let limits: SubagentRuntimeLimits
    private var active: [AgentRunID: Task<Void, Never>] = [:]
    private var queued: [(AgentRunID, @Sendable () async -> Void)] = []

    public init(limits: SubagentRuntimeLimits = SubagentRuntimeLimits()) { self.limits = limits }

    public func submit(runID: AgentRunID, operation: @escaping @Sendable () async -> Void) -> AgentRunStatus {
        if active.count < limits.maxConcurrentSubagents {
            start(runID, operation)
            return .starting
        }
        queued.append((runID, operation))
        return .queued
    }

    public func cancel(_ runID: AgentRunID) {
        if let index = queued.firstIndex(where: { $0.0 == runID }) {
            queued.remove(at: index)
            return
        }
        active.removeValue(forKey: runID)?.cancel()
    }

    public func cancelAll(_ runIDs: [AgentRunID]) {
        for runID in runIDs { cancel(runID) }
    }

    public func complete(_ runID: AgentRunID) {
        active.removeValue(forKey: runID)
        guard active.count < limits.maxConcurrentSubagents, !queued.isEmpty else { return }
        let next = queued.removeFirst()
        start(next.0, next.1)
    }

    public func snapshot() -> (active: [AgentRunID], queued: [AgentRunID]) {
        (Array(active.keys), queued.map{\.0})
    }

    private func start(_ runID: AgentRunID, _ operation: @escaping @Sendable () async -> Void) {
        active[runID] = Task { await operation() }
    }
}

enum AgentExecutionContext {
    @TaskLocal static var current: (sessionID: SessionID, runID: AgentRunID, rootSessionID: SessionID, parentSessionID: SessionID)?
}

```

// File: Sources/LingXiCore/Modules/Agent/SubagentToolService.swift

```

import Foundation
import LingXiProtocol

public enum SubagentTool {
    public static let definition = ToolDefinition(
        id: ToolID("subagent"),
        description: "Spawn, inspect, message, retrieve results from, or cancel an independent child Agent Session.",
        inputSchema: ToolInputSchema(properties: [
            "action": ToolInputProperty(type: .string, description: "spawn, status, message, result, or cancel", enumValues: ["spawn", "
↩ status", "message", "result", "cancel"]),
            "task": ToolInputProperty(type: .string, description: "Child task for spawn"),
            "title": ToolInputProperty(type: .string, description: "Optional child title"),
            "run_id": ToolInputProperty(type: .string, description: "Agent run ID"),
            "session_id": ToolInputProperty(type: .string, description: "Child session ID"),
            "content": ToolInputProperty(type: .string, description: "Additional child message"),
            "provider_id": ToolInputProperty(type: .string, description: "Requested provider ID"),
            "model_id": ToolInputProperty(type: .string, description: "Requested model ID"),
            "reasoning": ToolInputProperty(type: .string, description: "Requested reasoning mode")
        ], required: ["action"]),
        capability: ToolCapability([.projectRead])
    )
}

private struct SubagentToolArguments: Decodable {
    let action: String
    let task: String?
    let title: String?
    let runID: String?
    let sessionID: String?
    let content: String?
    let providerID: String?
    let modelID: String?
    let reasoning: String?
    let permissionProfile: String?
    let budgetProfile: String?
    let contextProfile: String?
    let maxSteps: Int?
    let timeoutSeconds: Int?

    enum CodingKeys: String, CodingKey {
        case action, task, title, content, reasoning
        case runID = "run_id"
        case sessionID = "session_id"
        case providerID = "provider_id"
        case modelID = "model_id"
        case permissionProfile = "permission_profile"
        case budgetProfile = "budget_profile"
        case contextProfile = "context_profile"
        case maxSteps = "max_steps"
        case timeoutSeconds = "timeout_seconds"
    }
}

var executionProfile: SubagentExecutionProfile? {
    guard permissionProfile != nil || budgetProfile != nil || contextProfile != nil || maxSteps != nil || timeoutSeconds != nil else {
        return nil
    }
    return SubagentExecutionProfile(
        permissionProfile: permissionProfile,
        budgetProfile: budgetProfile,
        contextProfile: contextProfile,
        maxSteps: maxSteps,
        timeoutSeconds: timeoutSeconds
    )
}
}

```

```

/// The Tool Plane calls this actor; it owns no Session state and delegates only through explicit runtime closures.
public actor SubagentToolService {
  private var spawn: (@Sendable (SessionID, AgentRunID, String, String?, ModelSelection?, SubagentExecutionProfile?, ToolCallID) async
    ↪ throws -> (SessionID, AgentRunInfo))?
  private var status: (@Sendable (AgentRunID, AgentRunID) async throws -> AgentRunInfo)?
  private var result: (@Sendable (AgentRunID, AgentRunID) async throws -> SubagentResult)?
  private var cancel: (@Sendable (AgentRunID, AgentRunID) async throws -> Void)?
  private var message: (@Sendable (SessionID, AgentRunID, String) async throws -> AgentRunInfo)?

  public init() {}

  public func bind(
    spawn: @escaping @Sendable (SessionID, AgentRunID, String, String?, ModelSelection?, SubagentExecutionProfile?, ToolCallID) async
    ↪ throws -> (SessionID, AgentRunInfo),
    status: @escaping @Sendable (AgentRunID, AgentRunID) async throws -> AgentRunInfo,
    result: @escaping @Sendable (AgentRunID, AgentRunID) async throws -> SubagentResult,
    cancel: @escaping @Sendable (AgentRunID, AgentRunID) async throws -> Void,
    message: @escaping @Sendable (SessionID, AgentRunID, String) async throws -> AgentRunInfo
  ) {
    self.spawn = spawn; self.status = status; self.result = result; self.cancel = cancel; self.message = message
  }

  public func execute(arguments: String, sessionID: SessionID, callID: ToolCallID) async throws -> String {
    guard let data = arguments.data(using: .utf8) else { throw CoreError(code: .toolArgumentInvalid, message: "subagent 参数不是 UTF-8") }
    ↪ }
    let input = try JSONDecoder().decode(SubagentToolArguments.self, from: data)
    let encoder = JSONEncoder(); encoder.outputFormatting = [.sortedKeys]
    let encode: @Sendable (any Encodable) throws -> String = { String(decoding: try encoder.encode(AnyEncodable($0)), as: UTF8.self) }
    switch input.action {
    case "spawn":
      guard let parentRunID = AgentExecutionContext.current?.runID,
        let task = (input.task ?? input.content)?.trimmingCharacters(in: .whitespacesAndNewlines),
        !task.isEmpty,
        let spawn else {
        throw CoreError(code: .toolArgumentInvalid, message: "spawn 需要当前 AgentRun 与 task")
      }
      let providerID = input.providerID?.trimmingCharacters(in: .whitespacesAndNewlines)
      let modelID = input.modelID?.trimmingCharacters(in: .whitespacesAndNewlines)
      let reasoning = input.reasoning?.trimmingCharacters(in: .whitespacesAndNewlines)
      let requestedSelection: ModelSelection?
      switch (providerID?.isEmpty == false ? providerID : nil, modelID?.isEmpty == false ? modelID : nil) {
      case (nil, nil) where reasoning?.isEmpty != false:
        requestedSelection = nil
      case let (providerID?, modelID?):
        requestedSelection = ModelSelection(providerID: providerID, modelID: modelID, reasoning: reasoning?.isEmpty == false ?
        ↪ reasoning : nil)
      default:
        throw CoreError(code: .toolArgumentInvalid, message: "subagent 的 provider_id 与 model_id 必须同时提供")
      }
      let title = input.title.flatMap { $0.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty ? nil : $0 }
      let (childSessionID, run) = try await spawn(sessionID, parentRunID, task, title, requestedSelection, input.executionProfile,
        ↪ callID)
      return try encode(SubagentSpawnResponse(childSessionID: childSessionID, run: run))
    case "status":
      guard let id = input.runID, let requester = AgentExecutionContext.current?.runID, let status else { throw CoreError(code: .
        ↪ toolArgumentInvalid, message: "status 需要 run_id 与当前 AgentRun") }
      return try encode(try await status(AgentRunID(id), requester))
    case "result":
      guard let id = input.runID, let requester = AgentExecutionContext.current?.runID, let result else { throw CoreError(code: .
        ↪ toolArgumentInvalid, message: "result 需要 run_id 与当前 AgentRun") }
      return try encode(try await result(AgentRunID(id), requester))
    case "cancel":
      guard let id = input.runID, let requester = AgentExecutionContext.current?.runID, let cancel else { throw CoreError(code: .
        ↪ toolArgumentInvalid, message: "cancel 需要 run_id 与当前 AgentRun") }

```

```

        try await cancel(AgentRunID(id), requester); return #"{"status":"cancelled"}"#
    case "message":
        guard let child = input.sessionID, let content = input.content?.trimmingCharacters(in: .whitespacesAndNewlines), !content.
        ↪ isEmpty, let parentRunID = AgentExecutionContext.current?.runID, let message else { throw CoreError(code: .toolArgumentInvalid,
        ↪ message: "message 需要 session_id、content 与当前 AgentRun") }
        return try encode(try await message(SessionID(child), parentRunID, content))
        default: throw CoreError(code: .toolArgumentInvalid, message: "未知 subagent action: \(input.action)")
    }
}
}

private struct SubagentSpawnResponse: Codable {
    let childSessionID: SessionID
    let run: AgentRunInfo
}

private struct AnyEncodable: Encodable {
    private let encodeValue: (Encoder) throws -> Void
    init(_ value: some Encodable) { encodeValue = value.encode }
    func encode(to encoder: Encoder) throws { try encodeValue(encoder) }
}

```

// File: Sources/LingXiCore/Modules/Context/L1ContextEngine.swift

```

import Foundation
import LingXiProtocol

/// L1 来源描述的是模型工作集的语义，不是任何 Provider 的角色类型。
public enum ContextSource: String, Sendable, Equatable, Hashable {
    case system
    case userMessage
    case assistantMessage
    case toolCall
    case toolResult
    case projectPage
    case derivedPage
}

public enum ContextRole: Sendable, Equatable {
    case system
    case user
    case assistant
    case tool
}

public struct ContextEntry: Sendable, Equatable {
    public let messageID: MessageID?
    public let role: ContextRole
    public let source: ContextSource
    public let part: SessionMessagePart
    public let page: ContextPage?

    public init(messageID: MessageID?, role: ContextRole, source: ContextSource, part: SessionMessagePart, page: ContextPage? = nil) {
        self.messageID = messageID
        self.role = role
        self.source = source
        self.part = part
        self.page = page
    }
}

public struct ContextMetrics: Sendable, Equatable {
    public let messageCount: Int
    public let partCount: Int
    public let characterCount: Int
}

```

```

public let sourceCounts: [ContextSource: Int]
public let sessionCharacterCount: Int
public let projectCharacterCount: Int
public let projectPageCount: Int
public let estimatedTokens: Int
public let derivedPageCount: Int
public let mandatoryTokens: Int
public let recentSessionTokens: Int
public let projectTokens: Int
public let derivedTokens: Int
public let liveToolBatchCount: Int
public let compactionGeneration: Int
}

/// 一次 inference 实际可见的不可变 L1 工作集。
public struct L1ContextSnapshot: Sendable, Equatable {
    public let sessionID: SessionID
    public let revision: UInt64
    public let entries: [ContextEntry]
    public let metrics: ContextMetrics

    public func modelMessages() -> [ModelMessage] {
        var result: [ModelMessage] = []
        var currentID: MessageID?
        var currentRole: ModelRole?
        var parts: [ModelContentPart] = []

        let renderedEntries = entries.flatMap { entry -> [ContextEntry] in
            guard entry.messageID == nil, entry.source == .system, case let .text(content) = entry.part,
                content.hasPrefix("Environment facts:\n"), let separator = content.range(of: "\n\n") else {
                return [entry]
            }
            let facts = String(content[..<separator.lowerBound])
            let instructions = String(content[separator.upperBound...])
            let instructionBlocks = instructions.components(separatedBy: "\n\n").filter { !$0.trimmingCharacters(in: .whitespacesAndNewlines)
↳ }.isEmpty }
            var fragments = instructionBlocks.isEmpty ? [instructions] : instructionBlocks
            if entries.first?.messageID == nil {
                fragments.append(facts)
            } else {
                fragments.insert(facts, at: 0)
            }
            return fragments.map { ContextEntry(messageID: nil, role: .system, source: .system, part: .text($0)) }
        }

        func appendCurrent() {
            if let currentRole { result.append(ModelMessage(role: currentRole, parts: parts)) }
        }

        for entry in renderedEntries {
            let role = Self.modelRole(entry.role)
            if currentID != entry.messageID || currentRole != role {
                appendCurrent()
                currentID = entry.messageID
                currentRole = role
                parts = []
            }
            parts.append(Self.modelPart(entry.part))
        }
        appendCurrent()
        return result
    }

    private static func modelRole(_ role: ContextRole) -> ModelRole {
        switch role {

```

```

        case .system: .system
        case .user: .user
        case .assistant: .assistant
        case .tool: .tool
    }
}

private static func modelPart(_ part: SessionMessagePart) -> ModelContentPart {
    switch part {
    case let .text(text): .text(text)
    case let .toolCall(call): .toolCall(call)
    case let .toolResult(result): .toolResult(result)
    }
}
}

/// L1 初始策略：保留已完成 Session 的有序结构化历史，明确排除 reasoning 与 transient stream。
public struct L1ContextPolicy: Sendable {
    public let systemContext: String?

    public init(systemContext: String? = nil) {
        self.systemContext = systemContext?.isEmpty == true ? nil : systemContext
    }
}

/// Context Engine 是 Session history 与当前模型工作集之间的正式边界。
public actor L1ContextEngine {
    private let policy: L1ContextPolicy
    private var revisions: [SessionID: UInt64] = [:]
    private var latest: [SessionID: L1ContextSnapshot] = [:]

    public init(policy: L1ContextPolicy = L1ContextPolicy()) {
        self.policy = policy
    }

    public func snapshot(for session: Session, projectPages: [ContextPage] = [], activeEntries: [ContextEntry]? = nil, systemContext: String
        ↪ ? = nil, estimatedTokens: Int = 0, mandatoryTokens: Int = 0, liveToolBatchCount: Int = 0, compactionGeneration: Int = 0) ->
        ↪ L1ContextSnapshot {
        let revision = (revisions[session.id] ?? 0) + 1
        revisions[session.id] = revision
        var entries = activeEntries ?? []
        if activeEntries == nil {
            if let system = systemContext ?? policy.systemContext {
                entries.append(ContextEntry(messageID: nil, role: .system, source: .system, part: .text(system)))
            }
        }
        for message in session.messages {
            for part in message.parts {
                let projectedPart: SessionMessagePart
                switch part {
                case let .toolResult(result):
                    projectedPart = .toolResult(ModelToolResultProjection.projectToolResult(result))
                default:
                    projectedPart = part
                }
                entries.append(ContextEntry(
                    messageID: message.id,
                    role: contextRole(message.role),
                    source: source(message.role, part),
                    part: projectedPart
                ))
            }
        }
        let toolContents = Set(session.messages.flatMap { message in
            message.parts.compactMap { if case let .toolResult(result) = $0 { result.content } else { nil } }
        })
    }
}

```

```

    })
    var seenPages = Set<String>()
    for page in projectPages where seenPages.insert("\${page.path}|\${page.hash}").inserted && !toolContents.contains(page.content) {
        entries.append(ContextEntry(messageID: MessageID("project:\${page.id}"), role: .system, source: .projectPage, part: .text("[
↪ Project context: \${page.path}:\${page.startLine}-\${page.endLine}]\n\${page.content}"), page: page))
    }
    let snapshot = L1ContextSnapshot(
        sessionID: session.id,
        revision: revision,
        entries: entries,
        metrics: metrics(entries, estimatedTokens: estimatedTokens, mandatoryTokens: mandatoryTokens, liveToolBatchCount:
↪ liveToolBatchCount, compactionGeneration: compactionGeneration, hasSystemContext: (systemContext ?? policy.systemContext) != nil)
    )
    latest[session.id] = snapshot
    return snapshot
}

public func latestSnapshot(for sessionID: SessionID) -> L1ContextSnapshot? {
    latest[sessionID]
}

public func reset(for sessionID: SessionID) {
    latest.removeValue(forKey: sessionID)
}

private func contextRole(_ role: MessageRole) -> ContextRole {
    switch role {
    case .user: .user
    case .assistant: .assistant
    case .tool: .tool
    }
}

private func source(_ role: MessageRole, _ part: SessionMessagePart) -> ContextSource {
    switch part {
    case .toolCall: .toolCall
    case .toolResult: .toolResult
    case .text:
        switch role {
        case .user: .userMessage
        case .assistant, .tool: .assistantMessage
        }
    }
}

public func entries(for session: Session, projectPages: [ContextPage] = [], systemContext: String? = nil, systemContextAtBeginning: Bool
↪ = true) -> [ContextEntry] {
    var entries: [ContextEntry] = []
    let systemEntry = (systemContext ?? policy.systemContext).map { ContextEntry(messageID: nil, role: .system, source: .system, part: .
↪ text($0)) }
    if systemContextAtBeginning, let systemEntry { entries.append(systemEntry) }
    for message in session.messages {
        for part in message.parts {
            let projectedPart: SessionMessagePart
            switch part {
            case let .toolResult(result):
                projectedPart = .toolResult(ModelToolResultProjection.projectToolResult(result))
            default:
                projectedPart = part
            }
            entries.append(ContextEntry(messageID: message.id, role: contextRole(message.role), source: source(message.role, part), part
↪ : projectedPart))
        }
    }
    let toolContents = Set(session.messages.flatMap { $0.parts.compactMap { if case let .toolResult(result) = $0 { result.content } else

```

```

    ↪ { nil } } })
    var seen = Set<String>()
    for page in projectPages where seen.insert("\(page.path)|\(page.hash)").inserted && !toolContents.contains(page.content) {
        entries.append(ContextEntry(messageID: MessageID("project:\(page.id)"), role: .system, source: .projectPage, part: .text("[
    ↪ Project context: \(page.path):\(page.startLine)-\(page.endLine)]\n\(page.content)"), page: page))
    }
    if !systemContextAtBeginning, let systemEntry { entries.append(systemEntry) }
    return entries
}

public func initialMandatoryTokens(task: String, estimator: any TokenEstimator = ConservativeTokenEstimator()) -> Int {
    var tokens = 0
    if let system = policy.systemContext, !system.isEmpty {
        tokens += estimator.estimate(text: system) + 4
    }
    tokens += estimator.estimate(text: task) + 4
    return tokens
}

private func metrics(_ entries: [ContextEntry], estimatedTokens: Int, mandatoryTokens: Int, liveToolBatchCount: Int,
    ↪ compactionGeneration: Int, hasSystemContext: Bool) -> ContextMetrics {
    var sourceCounts: [ContextSource: Int] = [:]
    var ids = Set<MessageID>()
    var characters = 0
    var sessionCharacters = 0
    var projectCharacters = 0
    for entry in entries {
        sourceCounts[entry.source, default: 0] += 1
        if let id = entry.messageID { ids.insert(id) }
        switch entry.part {
        case let .text(text): characters += text.count
        case let .toolCall(call): characters += call.arguments.count
        case let .toolResult(result):
            characters += result.content.count + (result.error?.message.count ?? 0)
        }
        let count: Int
        switch entry.part {
        case let .text(text): count = entry.page?.characterCount ?? text.count
        case let .toolCall(call): count = call.arguments.count
        case let .toolResult(result): count = result.content.count + (result.error?.message.count ?? 0)
        }
        if entry.source == .projectPage { projectCharacters += count } else { sessionCharacters += count }
    }
    let projectTokens = max(0, (projectCharacters + 2) / 3)
    let derivedCharacters = entries.filter { $0.source == .derivedPage }.reduce(0) { $0 + Self.characterCount(of: $1.part) }
    let derivedTokens = max(0, (derivedCharacters + 2) / 3)
    let effectiveTokens = estimatedTokens > 0 ? estimatedTokens : ConservativeTokenEstimator().estimate(entries: entries)
    return ContextMetrics(messageCount: ids.count + (hasSystemContext ? 1 : 0), partCount: entries.count, characterCount: characters,
    ↪ sourceCounts: sourceCounts, sessionCharacterCount: sessionCharacters, projectCharacterCount: projectCharacters, projectPageCount:
    ↪ sourceCounts[.projectPage, default: 0], estimatedTokens: effectiveTokens, derivedPageCount: sourceCounts[.derivedPage, default: 0],
    ↪ mandatoryTokens: mandatoryTokens, recentSessionTokens: max(0, effectiveTokens - projectTokens - derivedTokens - mandatoryTokens),
    ↪ projectTokens: projectTokens, derivedTokens: derivedTokens, liveToolBatchCount: liveToolBatchCount, compactionGeneration:
    ↪ compactionGeneration)
}

private static func characterCount(of part: SessionMessagePart) -> Int {
    switch part { case let .text(text): text.count; case let .toolCall(call): call.arguments.count; case let .toolResult(result): result
    ↪ .content.count + (result.error?.message.count ?? 0) }
}
}

```

// File: Sources/LingXiCore/Modules/Context/L2WorkingSetPolicy.swift

import Foundation

```

        model: request.model.rawValue,
        wireAdapter: config.wireProtocol.rawValue,
        url: urlRequest.url?.absoluteString ?? config.anthropicMessagesURL.absoluteString,
        httpMethod: urlRequest.httpMethod ?? "POST",
        requestFields: requestFields,
        toolSchemas: toolSchemas,
        capabilityProjection: capabilityProjection,
        responseBody: body
    )

    let error = OpenAICompatibleProvider.httpError(status: response.statusCode, requestID: requestID, diagnostics: diagnostics)
    throw ProviderRateLimitError.from(statusCode: response.statusCode, headers: response.headers, body: body, underlying: error)
}

return AsyncThrowingStream { continuation in
    if let requestID = ProviderTraceSanitizer.requestID(requestID) {
        continuation.yield(.providerRequestID(requestID))
    }

    let pump = Task { await Pump(source: response.body, continuation: continuation, request: request, prior: prior, provenance:
↪ provenance).run() }
    continuation.onTermination = { @Sendable _ in
        pump.cancel()
    }
}
}

public func endExecution(_ executionID: AgentRunID) async {
    await provenance.remove(executionID)
}

public func makeURLRequest(_ request: ModelRequest) throws -> URLRequest {
    try makeURLRequest(request, continuation: nil)
}

private func makeURLRequest(_ request: ModelRequest, continuation: ProviderContinuation?) throws -> URLRequest {
    guard request.reasoning == nil else {
        throw CoreError(code: .provider, message: "Anthropic reasoning configuration is not implemented")
    }

    var result = URLRequest(url: config.anthropicMessagesURL)
    result.httpMethod = "POST"
    if let timeout = request.overallTimeoutSeconds { result.timeoutInterval = timeout }
    result.setValue("application/json", forHTTPHeaderField: "Content-Type")
    result.setValue("2023-06-01", forHTTPHeaderField: "anthropic-version")
    switch config.authentication {
    case .none: break
    case let .bearer(secret): result.setValue("Bearer \(secret)", forHTTPHeaderField: "Authorization")
    case let .header(name, value): result.setValue(value, forHTTPHeaderField: name)
    }

    for (name, value) in config.requiredHeaders {
        result.setValue(value, forHTTPHeaderField: name)
    }

    result.httpBody = try Self.makeRequestBody(request, maxOutputTokens: config.maxOutputTokens ?? 4_096, continuation: continuation)
    return result
}

public static func makeRequestBody(_ request: ModelRequest, maxOutputTokens: Int = 4_096) throws -> Data {
    try makeRequestBody(request, maxOutputTokens: maxOutputTokens, continuation: nil)
}

private static func makeRequestBody(_ request: ModelRequest, maxOutputTokens: Int, continuation: ProviderContinuation?) throws -> Data {
    let orderedTools: [ToolDefinition]
    let system: String?
    if let plan = request.cachePlan {
        system = plan.immutableBase.systemPrompt
        orderedTools = plan.immutableBase.coreTools + plan.appendOnlyContext.dynamicTools
    } else {
        let messageSystem = request.messages.filter { $0.role == .system }.map(\.content).filter { !$0.isEmpty }

```

```

        system = ([request.system].compactMap { $0 } + messageSystem).joined(separator: "\n\n")
        let coreIDs = ToolRuntime.coreToolIDs
        let core = request.tools.filter { coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
        let dynamic = request.tools.filter { !coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
        orderedTools = core + dynamic
    }

    let messages = try request.messages.compactMap { message -> RequestBody.Message? in
        switch message.role {
        case .system:
            return nil
        case .user:
            return RequestBody.Message(role: "user", content: message.parts.compactMap { part in
                if case let .text(text) = part { return .text(text) }
                return nil
            })
        case .assistant:
            return RequestBody.Message(role: "assistant", content: try message.parts.map { part in
                switch part {
                case let .text(text): return .text(text)
                case let .toolCall(call):
                    guard let data = call.arguments.data(using: .utf8),
                          let input = try? JSONDecoder().decode(JSONValue.self, from: data),
                          case .object = input
                    else { throw CoreError(code: .provider, message: "Anthropic tool input must be a JSON object") }
                    return .toolUse(id: continuation?.externalCallID(for: call.callID) ?? call.callID.rawValue, name: call.toolID,
↳ rawValue, input: input)
                case .toolResult:
                    throw CoreError(code: .provider, message: "Anthropic assistant message cannot contain tool results")
                }
            })
        case .tool:
            return RequestBody.Message(role: "user", content: message.parts.compactMap { part in
                guard case let .toolResult(result) = part else { return nil }
                let projected = ModelToolResultProjection.project(result)
                return .toolResult(id: continuation?.externalCallID(for: projected.callID) ?? projected.callID.rawValue, content:
↳ projected.content, isError: !projected.success)
            })
        }
    }

    let enableExplicitCache = request.cachePlan?.capabilities?.explicitCacheControlSupported ?? false
    let coreCount = request.cachePlan?.immutableBase.coreTools.count ?? 0
    let tools: [RequestBody.Tool]? = orderedTools.isEmpty ? nil : orderedTools.enumerated().map { index, def in
        let isBreakpoint = enableExplicitCache && index == (coreCount > 0 ? coreCount - 1 : 0)
        return RequestBody.Tool(def, cacheControl: isBreakpoint ? RequestBody.CacheControl() : nil)
    }

    let encoder = JSONEncoder()
    encoder.outputFormatting = [.sortedKeys]
    return try encoder.encode(RequestBody(
        model: request.model.rawValue,
        maxTokens: max(1, maxOutputTokens),
        stream: true,
        system: (system?.isEmpty ?? true) ? nil : system,
        messages: messages,
        tools: tools
    ))
}

private final class Pump: Sendable {
    private let source: AsyncThrowingStream<Data, Error>
    private let continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation
    private let request: ModelRequest
    private let prior: ProviderContinuation?

```

```

private let provenance: ProviderProvenanceStore

init(source: AsyncThrowingStream<Data, Error>, continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation, request:
↳ ModelRequest, prior: ProviderContinuation?, provenance: ProviderProvenanceStore) {
    self.source = source
    self.continuation = continuation
    self.request = request
    self.prior = prior
    self.provenance = provenance
}

func run() async {
    var lines = SSEDecoder()
    var decoder = AnthropicSSEDecoder(requestID: request.requestID)
    var terminal = false
    var failed = false
    continuation.yield(.started)
    do {
        outer: for try await chunk in source {
            for line in lines.feed(chunk) {
                if try emit(line, decoder: &decoder, terminal: &terminal, failed: &failed) { break outer }
            }
        }
        if !terminal {
            for line in lines.flushPending() {
                _ = try emit(line, decoder: &decoder, terminal: &terminal, failed: &failed)
            }
        }
        if Task.isCancelled || failed { continuation.finish(); return }
        guard terminal else { throw CoreError(code: .modelStream, message: "Anthropic SSE ended unexpectedly") }
        let combinedReferences = (prior?.references ?? []).filter { priorRef in !decoder.references.contains { $0.domainCallID ==
↳ priorRef.domainCallID } } + decoder.references
        if !combinedReferences.isEmpty {
            try await provenance.record(ProviderContinuation(requestID: request.requestID, executionID: request.executionID, wire: .
↳ anthropicMessages, references: combinedReferences))
        }
        continuation.finish()
    } catch let error as CoreError {
        if Task.isCancelled { continuation.finish(); return }
        continuation.yield(.failed(error))
        continuation.finish()
    } catch {
        if Task.isCancelled { continuation.finish(); return }
        continuation.yield(.failed(CoreError(code: .modelStream, message: "Anthropic SSE connection interrupted")))
        continuation.finish()
    }
}

private func emit(_ line: String, decoder: inout AnthropicSSEDecoder, terminal: inout Bool, failed: inout Bool) throws -> Bool {
    let trimmed = line.trimmingCharacters(in: .whitespaces)
    guard trimmed.hasPrefix("data:") else { return false }
    let payload = String(trimmed.dropFirst("data:".count)).trimmingCharacters(in: .whitespaces)
    if payload == "[DONE]" { return false }
    if let data = payload.data(using: .utf8),
        let object = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
        let type = object["type"] as? String {
        terminal = terminal || type == "message_stop" || type == "error"
    }
    for event in try decoder.consume(payload) {
        if case .failed = event { failed = true }
        continuation.yield(event)
    }
    return terminal
}
}

```

```

}

public struct AnthropicSSEDecoder {
    private struct PartialTool {
        var externalID: String
        var domainID: ToolCallID
        var name: String
        var arguments = ""
    }

    private var tools: [Int: PartialTool] = [:]
    private let requestID: ModelRequestID
    private var pendingFinishReason: ModelFinishReason?
    public private(set) var references: [ProviderToolCallReference] = []

    public init(requestID: ModelRequestID = ModelRequestID()) { self.requestID = requestID }

    public mutating func consume(_ payload: String) throws -> [ModelEvent] {
        guard let data = payload.data(using: .utf8),
              let object = try JSONSerialization.jsonObject(with: data) as? [String: Any],
              let type = object["type"] as? String
        else { throw CoreError(code: .modelStream, message: "Anthropic SSE JSON parsing failed") }

        switch type {
        case "message_start":
            let message = object["message"] as? [String: Any]
            return usage(message?["usage"] as? [String: Any]).map { [.usage($0)] } ?? []
        case "content_block_start":
            guard let index = object["index"] as? Int, let block = object["content_block"] as? [String: Any] else { return [] }
            switch block["type"] as? String {
            case "text": return (block["text"] as? String).flatMap { $0.isEmpty ? nil : [.textDelta($0)] } ?? []
            case "thinking": return []
            case "tool_use":
                guard let id = block["id"] as? String, let name = block["name"] as? String else {
                    throw CoreError(code: .modelStream, message: "Anthropic tool use is missing identity")
                }
                let domainID = ToolCallID("lingxi:\(requestID.rawValue):\(\index)")
                tools[index] = PartialTool(externalID: id, domainID: domainID, name: name)
                return [.toolCallStarted(callID: domainID, toolID: ToolID(name))]
            default: return []
            }
        case "content_block_delta":
            guard let index = object["index"] as? Int, let delta = object["delta"] as? [String: Any] else { return [] }
            switch delta["type"] as? String {
            case "text_delta": return (delta["text"] as? String).map { [.textDelta($0)] } ?? []
            case "thinking_delta": return []
            case "input_json_delta":
                guard var tool = tools[index], let fragment = delta["partial_json"] as? String else { return [] }
                tool.arguments += fragment
                tools[index] = tool
                return [.toolCallDelta(callID: tool.domainID, arguments: fragment)]
            default: return []
            }
        case "content_block_stop":
            guard let index = object["index"] as? Int, let tool = tools.removeValue(forKey: index) else { return [] }
            guard let data = tool.arguments.data(using: .utf8),
                  (try? JSONSerialization.jsonObject(with: data)) is [String: Any]
            else { throw CoreError(code: .modelStream, message: "Anthropic tool input is not a JSON object") }
            references.append(ProviderToolCallReference(wire: .anthropicMessages, domainCallID: tool.domainID, externalCallID: tool.
↵ externalID))
            return [.toolCallCompleted(ToolCall(callID: tool.domainID, toolID: ToolID(tool.name), arguments: tool.arguments))]
        case "message_delta":
            var events: [ModelEvent] = []
            if let value = usage(object["usage"] as? [String: Any]) { events.append(.usage(value)) }
            let delta = object["delta"] as? [String: Any]

```

```

        if let reason = finishReason(delta?["stop_reason"] as? String) { pendingFinishReason = reason }
        return events
    case "message_stop":
        return [.completed(pendingFinishReason ?? .unknown)]
    case "error":
        return [.failed(CoreError(code: .modelStream, message: "Anthropic SSE returned an error"))]
    default:
        return []
    }
}

private func usage(_ raw: [String: Any]?) -> ModelUsage? {
    guard let raw else { return nil }
    return ModelUsage(
        inputTokens: raw["input_tokens"] as? Int,
        outputTokens: raw["output_tokens"] as? Int,
        reasoningTokens: nil,
        cacheReadTokens: raw["cache_read_input_tokens"] as? Int,
        cacheWriteTokens: raw["cache_creation_input_tokens"] as? Int
    )
}

private func finishReason(_ raw: String?) -> ModelFinishReason? {
    switch raw {
    case "end_turn", "stop_sequence": .stop
    case "max_tokens": .maxTokens
    case "tool_use": .toolCalls
    case "refusal": .contentFilter
    case nil, "": nil
    default: .unknown
    }
}

private struct RequestBody: Encodable {
    struct CacheControl: Encodable {
        let type: String
        init(type: String = "ephemeral") { self.type = type }
    }

    struct Message: Encodable { let role: String; let content: [Content] }
    struct Tool: Encodable {
        let name: String
        let description: String
        let inputSchema: JSONValue
        let cacheControl: CacheControl?

        init(_ definition: ToolDefinition, cacheControl: CacheControl? = nil) {
            name = definition.name
            description = definition.description
            inputSchema = definition.rawInputSchema ?? .object([
                "type": .string("object"),
                "properties": .object(definition.inputSchema.properties.mapValues { .object(["type": .string($0.type.rawValue), "description"
↪ ": .string($0.description)]) })),
                "required": .array(definition.inputSchema.required.map(JSONValue.string)),
                "additionalProperties": .bool(false),
            ])
            self.cacheControl = cacheControl
        }
    }

    enum CodingKeys: String, CodingKey {
        case name, description
        case inputSchema = "input_schema"
        case cacheControl = "cache_control"
    }
}

```

```

}

enum Content: Encodable {
    case text(String)
    case toolUse(id: String, name: String, input: JSONValue)
    case toolResult(id: String, content: String, isError: Bool)

    enum Keys: String, CodingKey { case type, text, id, name, input, toolUseID = "tool_use_id", content, isError = "is_error" }
    func encode(to encoder: Encoder) throws {
        var values = encoder.container(keyedBy: Keys.self)
        switch self {
        case let .text(text):
            try values.encode("text", forKey: .type); try values.encode(text, forKey: .text)
        case let .toolUse(id, name, input):
            try values.encode("tool_use", forKey: .type); try values.encode(id, forKey: .id); try values.encode(name, forKey: .name);
            ↪ try values.encode(input, forKey: .input)
        case let .toolResult(id, content, isError):
            try values.encode("tool_result", forKey: .type); try values.encode(id, forKey: .toolUseID); try values.encode(content,
            ↪ forKey: .content); try values.encode(isError, forKey: .isError)
        }
    }
}

let model: String
let maxTokens: Int
let stream: Bool
let system: String?
let messages: [Message]
let tools: [Tool]?

enum CodingKeys: String, CodingKey { case model, stream, system, messages, tools; case maxTokens = "max_tokens" }
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderRateScheduler.swift

```

import Foundation
import LingXiProtocol

public struct ProviderRetryPolicy: Sendable, Equatable, Codable {
    public let maxRetries: Int
    public let initialDelayMilliseconds: Int
    public let maxDelayMilliseconds: Int
    public let jitterRatio: Double

    public init(maxRetries: Int = 5, initialDelayMilliseconds: Int = 2_000, maxDelayMilliseconds: Int = 30_000, jitterRatio: Double = 0.25)
    ↪ {
        self.maxRetries = max(0, maxRetries)
        self.initialDelayMilliseconds = max(0, initialDelayMilliseconds)
        self.maxDelayMilliseconds = max(initialDelayMilliseconds, maxDelayMilliseconds)
        self.jitterRatio = min(1, max(0, jitterRatio))
    }

    private enum CodingKeys: String, CodingKey { case maxRetries, initialDelayMilliseconds, maxDelayMilliseconds, jitterRatio }

    public init(from decoder: Decoder) throws {
        let values = try decoder.container(keyedBy: CodingKeys.self)
        self.init(
            maxRetries: try values.decodeIfPresent(Int.self, forKey: .maxRetries) ?? 5,
            initialDelayMilliseconds: try values.decodeIfPresent(Int.self, forKey: .initialDelayMilliseconds) ?? 2_000,
            maxDelayMilliseconds: try values.decodeIfPresent(Int.self, forKey: .maxDelayMilliseconds) ?? 30_000,
            jitterRatio: try values.decodeIfPresent(Double.self, forKey: .jitterRatio) ?? 0.25
        )
    }
}

public struct ProviderRateLimits: Sendable, Equatable, Codable {

```

```

public let tpm: Int?
public let rpm: Int?
public let maxConcurrentRequests: Int?
public let retryPolicy: ProviderRetryPolicy

public init(tpm: Int? = nil, rpm: Int? = nil, maxConcurrentRequests: Int? = nil, retryPolicy: ProviderRetryPolicy = ProviderRetryPolicy
↳ ()) {
    self.tpm = tpm.flatMap { $0 > 0 ? $0 : nil }
    self.rpm = rpm.flatMap { $0 > 0 ? $0 : nil }
    self.maxConcurrentRequests = maxConcurrentRequests.flatMap { $0 > 0 ? $0 : nil }
    self.retryPolicy = retryPolicy
}

private enum CodingKeys: String, CodingKey { case tpm, rpm, maxConcurrentRequests, retryPolicy }

public init(from decoder: Decoder) throws {
    let values = try decoder.container(keyedBy: CodingKeys.self)
    self.init(
        tpm: try values.decodeIfPresent(Int.self, forKey: .tpm),
        rpm: try values.decodeIfPresent(Int.self, forKey: .rpm),
        maxConcurrentRequests: try values.decodeIfPresent(Int.self, forKey: .maxConcurrentRequests),
        retryPolicy: try values.decodeIfPresent(ProviderRetryPolicy.self, forKey: .retryPolicy) ?? ProviderRetryPolicy()
    )
}

public struct ProviderRateLimitError: Error, Sendable {
    public let statusCode: Int
    public let retryAfter: Duration?
    public let underlying: CoreError
    public let classified: ClassifiedProviderError

    public init(statusCode: Int, retryAfter: Duration? = nil, underlying: CoreError, classified: ClassifiedProviderError? = nil) {
        self.statusCode = statusCode
        self.retryAfter = retryAfter
        self.underlying = underlying
        self.classified = classified ?? ProviderErrorClassifier.classify(statusCode: statusCode, headers: [:], body: nil, underlying:
↳ underlying)
    }

    public static func from(statusCode: Int, headers: [String: String], body: String, underlying: CoreError) -> Error {
        let classified = ProviderErrorClassifier.classify(statusCode: statusCode, headers: headers, body: body, underlying: underlying)
        if classified.category.isRetryable {
            return ProviderRateLimitError(
                statusCode: statusCode,
                retryAfter: classified.retryAfter,
                underlying: underlying,
                classified: classified
            )
        } else {
            // 不可重试的永久错误，立即熔断并给出结构化诊断与排障建议
            var msg = classified.userFacingSummary
            let diag = classified.diagnostics ?? underlying.message
            if !diag.isEmpty && !diag.contains(classified.userFacingSummary) {
                msg += "\n[Wire Diagnostics]\n" + diag
            }
            return CoreError(code: .provider, message: msg)
        }
    }
}

public struct ProviderRateMetrics: Sendable, Equatable {
    public let retryCount: Int
    public let rateWaitMilliseconds: Int
    public let rateLimit429Count: Int

```

```

}

actor ProviderRateScheduler {
    static let shared = ProviderRateScheduler()

    private struct Key: Hashable {
        let provider: String
        let model: String
        let account: String?
    }

    private struct Workload {
        let requestID: ModelRequestID
        let timestamp: Date
        let tokens: Int
    }

    private var workloads: [Key: [Workload]] = [:]
    private var activeRequestIDs: [Key: Set<ModelRequestID>] = [:]
    private var blockedUntil: [Key: Date] = [:]
    private var metrics: [ModelRequestID: ProviderRateMetrics] = [:]

    func admit(endpoint: ResolvedModelEndpoint, requestID: ModelRequestID, estimatedTokens: Int) async throws {
        let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
        let limits = endpoint.rateLimits
        let clock = ContinuousClock()
        let started = clock.now
        var iterations = 0
        while true {
            try Task.checkCancellation()
            prune(key)
            if let blocked = blockedUntil[key], blocked > .now {
                try await Task.sleep(for: .milliseconds(max(1, Int(blocked.timeIntervalSinceNow * 1_000))))
                continue
            }
            let current = workloads[key] ?? []
            let concurrent = activeRequestIDs[key]?.count ?? 0
            let hasConcurrencyCapacity = concurrent < (limits.maxConcurrentRequests ?? .max)
            let hasRpmCapacity = current.count < (limits.rpm ?? .max)
            let hasTpmCapacity = canAdmitTokens(current, estimate: estimatedTokens, limit: limits.tpm)

            if hasConcurrencyCapacity && hasRpmCapacity && hasTpmCapacity {
                workloads[key, default: []].append(Workload(requestID: requestID, timestamp: .now, tokens: estimatedTokens))
                activeRequestIDs[key, default: []].insert(requestID)
                let elapsed = started.duration(to: clock.now).components
                let waited = elapsed.seconds * 1_000 + elapsed.attoseconds / 1_000_000_000_000_000
                if waited > 0 { addWait(requestID, milliseconds: Int(waited)) }
                return
            }

            iterations += 1
            // 积极放行与自愈: 当有空余并发槽位且无服务端 429 锁定, 但仅因本地 TPM 累加虚高被阻挡时, 在轻量交错退避 (<=40ms) 后积极放行, 绝不因
            ↪ 本地累加虚高阻塞多轮推理
            if hasConcurrencyCapacity && hasRpmCapacity && iterations > 2 && blockedUntil[key] == nil {
                workloads[key, default: []].append(Workload(requestID: requestID, timestamp: .now, tokens: estimatedTokens))
                activeRequestIDs[key, default: []].insert(requestID)
                let elapsed = started.duration(to: clock.now).components
                let waited = elapsed.seconds * 1_000 + elapsed.attoseconds / 1_000_000_000_000_000
                if waited > 0 { addWait(requestID, milliseconds: Int(waited)) }
                return
            }
        }

        if iterations > 30 && concurrent > 0 {
            activeRequestIDs[key]?.removeAll()
            continue
        }

        try await Task.sleep(for: waitDuration(current, concurrent: concurrent, limits: limits, estimate: estimatedTokens))
    }

```

```

    }
}

public func activeRequests(for endpoint: ResolvedModelEndpoint) -> Int {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    return activeRequestIDs[key]?.count ?? 0
}

public func release(endpoint: ResolvedModelEndpoint) {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    if var set = activeRequestIDs[key], !set.isEmpty {
        set.remove(set.first!)
        activeRequestIDs[key] = set
    }
}

public func release(requestID: ModelRequestID, endpoint: ResolvedModelEndpoint) {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    activeRequestIDs[key]?.remove(requestID)
}

public func cancel(requestID: ModelRequestID, endpoint: ResolvedModelEndpoint) {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    activeRequestIDs[key]?.remove(requestID)
    workloads[key]?.removeAll { $0.requestID == requestID }
}

public func reset(endpoint: ResolvedModelEndpoint? = nil) {
    if let endpoint {
        let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
        activeRequestIDs.removeValue(forKey: key)
        workloads.removeValue(forKey: key)
        blockedUntil.removeValue(forKey: key)
    } else {
        activeRequestIDs.removeAll()
        workloads.removeAll()
        blockedUntil.removeAll()
    }
}

func recordRetry(requestID: ModelRequestID) {
    let current = metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
    metrics[requestID] = ProviderRateMetrics(retryCount: current.retryCount + 1, rateWaitMilliseconds: current.rateWaitMilliseconds,
    ↪ rateLimit429Count: current.rateLimit429Count)
}

func recordRateLimit(endpoint: ResolvedModelEndpoint, requestID: ModelRequestID, cooldown: Duration) {
    let current = metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
    metrics[requestID] = ProviderRateMetrics(retryCount: current.retryCount, rateWaitMilliseconds: current.rateWaitMilliseconds,
    ↪ rateLimit429Count: current.rateLimit429Count + 1)
    let components = cooldown.components
    let seconds = Double(components.seconds) + Double(components.attoseconds) / 1_000_000_000_000_000
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    blockedUntil[key] = max(blockedUntil[key] ?? .distantPast, .now.addingTimeInterval(seconds))
}

func recordWait(requestID: ModelRequestID, duration: Duration) {
    let components = duration.components
    let milliseconds = Int(components.seconds * 1_000 + components.attoseconds / 1_000_000_000_000_000)
    if milliseconds > 0 { addWait(requestID, milliseconds: milliseconds) }
}

func recordUsage(endpoint: ResolvedModelEndpoint, requestID: ModelRequestID, usage: ModelUsage) {
    let actualTokens = [usage.inputTokens, usage.outputTokens, usage.reasoningTokens].compactMap { $0 }.reduce(0, +)
    guard actualTokens > 0 else { return }

```

```

        let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
        workloads[key] = workloads[key, default: []].map {
            $0.requestID == requestID ? Workload(requestID: $0.requestID, timestamp: $0.timestamp, tokens: actualTokens) : $0
        }
    }

func metrics(for requestID: ModelRequestID) -> ProviderRateMetrics {
    metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
}

private func addWait(_ requestID: ModelRequestID, milliseconds: Int) {
    let current = metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
    metrics[requestID] = ProviderRateMetrics(retryCount: current.retryCount, rateWaitMilliseconds: current.rateWaitMilliseconds +
        ↪ milliseconds, rateLimit429Count: current.rateLimit429Count)
}

private func prune(_ key: Key) {
    let cutoff = Date.now.addingTimeInterval(-60)
    workloads[key] = workloads[key, default: []].filter { $0.timestamp > cutoff }
    if (blockedUntil[key] ?? .distantPast) <= .now { blockedUntil[key] = nil }
}

private func canAdmitTokens(_ workloads: [Workload], estimate: Int, limit: Int?) -> Bool {
    guard let limit else { return true }
    return estimate > limit || workloads.reduce(0) { $0 + $1.tokens } + estimate <= limit
}

private func waitDuration(_ workloads: [Workload], concurrent: Int, limits: ProviderRateLimits, estimate: Int) -> Duration {
    if let maximum = limits.maxConcurrentRequests, concurrent >= maximum { return .milliseconds(10) }
    if let rpm = limits.rpm, workloads.count >= rpm, let oldest = workloads.map(\.timestamp).min() {
        let delayMs = min(20, max(1, Int(oldest.addingTimeInterval(60).timeIntervalSinceNow * 1_000)))
        return .milliseconds(delayMs)
    }
    if let tpm = limits.tpm, estimate <= tpm {
        var total = workloads.reduce(0) { $0 + $1.tokens }
        for workload in workloads.sorted(by: { $0.timestamp < $1.timestamp }) where total + estimate > tpm {
            total -= workload.tokens
            if total + estimate <= tpm {
                let delayMs = min(20, max(1, Int(workload.timestamp.addingTimeInterval(60).timeIntervalSinceNow * 1_000)))
                return .milliseconds(delayMs)
            }
        }
    }
    return .milliseconds(10)
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderErrorClassifier.swift

```

import Foundation
import LingXiProtocol

/// 标准化的模型提供商与上游网关错误分类。
public enum ProviderErrorCategory: String, Sendable, Codable, Equatable, CaseIterable {
    // MARK: - 瞬态错误（可自动指数退避重试）
    /// HTTP 429 或 TPM/RPM 配额耗尽
    case rateLimited
    /// HTTP 503 / 529 提供商服务暂时过载
    case serverOverloaded
    /// HTTP 408 / 504 / 522 / 524 或首字等待超时
    case gatewayTimeout
    /// HTTP 502 / 520 / 521 / 523 网关或反向代理异常
    case badGateway
    /// HTTP 500 提供商服务内部错误
    case internalServerError
}

```

```
/// 底层传输中断、连接重置、DNS 失败
case networkFailure

// MARK: - 永久性错误（不可重试，立即熔断）
/// HTTP 401: API Key / Access Token 无效或已过期
case authFailure
/// HTTP 402 / insufficient_quota: 余额不足或额度耗尽
case quotaExhausted
/// HTTP 403: 访问被拒绝、区域限制或账号风控
case accessForbidden
/// HTTP 404 / model_not_found: 模型、接口或资源不存在
case modelNotFound
/// HTTP 405: 请求方法不支持
case methodNotAllowed
/// HTTP 409: 资源状态冲突
case conflict
/// HTTP 413: 请求体或上下文过大超出单次限制
case payloadTooLarge
/// HTTP 415: 媒体格式或 Content-Type 不支持
case unsupportedMediaType
/// HTTP 400 (context_length_exceeded): 模型上下文长度超限
case contextExceeded
/// HTTP 400 / 422: 参数错误或不符合接口 Schema
case invalidRequest
/// HTTP 423: 账号或资源被临时锁定
case locked
/// content_filter / safety: 触发安全审查与敏感词拦截
case contentFiltered
/// HTTP 501: 功能未实现
case notImplemented
/// 未知异常
case unknown

/// 是否属于瞬态可重试错误。
public var isRetryable: Bool {
    switch self {
        case .rateLimited, .serverOverloaded, .gatewayTimeout, .badGateway, .internalServerError, .networkFailure:
            return true
        case .authFailure, .quotaExhausted, .accessForbidden, .modelNotFound, .methodNotAllowed,
            .conflict, .payloadTooLarge, .unsupportedMediaType, .contextExceeded, .invalidRequest,
            .locked, .contentFiltered, .notImplemented, .unknown:
            return false
    }
}

/// 用户可理解的中文诊断描述。
public var userDescription: String {
    switch self {
        case .rateLimited:
            return "上游请求速率超限 (429)"
        case .serverOverloaded:
            return "上游服务暂时过载 (529/503)"
        case .gatewayTimeout:
            return "上游网关或响应超时 (504/408)"
        case .badGateway:
            return "上游网关异常 (502)"
        case .internalServerError:
            return "上游服务内部错误 (500)"
        case .networkFailure:
            return "网络连接中断或握手失败"
        case .authFailure:
            return "API Key / Access Token 无效或已过期 (401)"
        case .quotaExhausted:
            return "账户余额不足或额度耗尽 (402)"
        case .accessForbidden:
```

```

        return "访问被拒绝 (403), 可能是账号风控或区域限制"
    case .modelNotFound:
        return "模型或接口资源不存在 (404)"
    case .methodNotAllowed:
        return "HTTP 请求方法不支持 (405)"
    case .conflict:
        return "资源状态冲突 (409)"
    case .payloadTooLarge:
        return "请求体或上下文过大超出单次限制 (413)"
    case .unsupportedMediaType:
        return "媒体格式不支持 (415)"
    case .contextExceeded:
        return "模型上下文长度超限 (400)"
    case .invalidRequest:
        return "请求参数错误或不符合接口规范 (400/422)"
    case .locked:
        return "账号或资源被临时锁定 (423)"
    case .contentFiltered:
        return "触发提供商内容安全审查风控拦截"
    case .notImplemented:
        return "上游接口未实现此功能 (501)"
    case .unknown:
        return "上游模型服务异常"
    }
}

```

/// 针对永久熔断错误给用户的行动建议。

```

public var suggestedAction: String? {
    switch self {
    case .authFailure:
        return "请检查提供商配置中的 API Key 是否有效或过期"
    case .quotaExhausted:
        return "请检查账户余额并前往服务商后台充值, 或切换至其他可用模型"
    case .accessForbidden:
        return "请检查当前网络地区或账号权限设置"
    case .modelNotFound:
        return "请检查模型名称配置是否正确, 或当前 API Key 是否有该模型权限"
    case .contextExceeded:
        return "上下文已达模型上限, 请执行 /compact 压缩会话或新建会话"
    case .payloadTooLarge:
        return "单次输入内容过大, 请精简输入或附件后重试"
    case .contentFiltered:
        return "请调整输入内容避免触碰提供商合规策略"
    case .invalidRequest:
        return "请求参数或工具调用定义不符合上游规范, 请检查配置"
    case .locked:
        return "账号或资源被锁定, 请登录服务商平台解锁"
    default:
        return nil
    }
}
}

```

/// 标准化分类后的模型提供商错误。

```

public struct ClassifiedProviderError: Error, Sendable {
    public let category: ProviderErrorCategory
    public let statusCode: Int?
    public let errorType: String?
    public let serverMessage: String?
    public let retryAfter: Duration?
    public let underlying: Error?
    public let diagnostics: String?

    public init(
        category: ProviderErrorCategory,

```

```

        statusCode: Int? = nil,
        errorType: String? = nil,
        serverMessage: String? = nil,
        retryAfter: Duration? = nil,
        underlying: Error? = nil,
        diagnostics: String? = nil
    ) {
        self.category = category
        self.statusCode = statusCode
        self.errorType = errorType
        self.serverMessage = serverMessage
        self.retryAfter = retryAfter
        self.underlying = underlying
        self.diagnostics = diagnostics
    }

    /// 面向用户的清晰诊断概要。
    public var userFacingSummary: String {
        var parts: [String] = []
        if let code = statusCode {
            parts.append("[\($code)]")
        }
        parts.append(category.userDescription)
        if let msg = serverMessage, !msg.isEmpty {
            parts.append("· \($msg)")
        }
        if let action = category.suggestedAction {
            parts.append("\($action)")
        }
        return parts.joined(separator: " ")
    }
}

/// 模型错误分类器。
public enum ProviderErrorClassifier {
    /// 对 HTTP 状态码、响应体及底层错误进行全面分类。
    public static func classify(
        statusCode: Int?,
        headers: [String: String] = [:],
        body: String? = nil,
        underlying: Error? = nil,
        diagnostics: String? = nil
    ) -> ClassifiedProviderError {
        let (extractedType, extractedMsg) = parseJSONError(body)
        let normalizedBody = (body ?? "").lowercased()
        let normalizedMsg = (extractedMsg ?? "").lowercased()
        let normalizedType = (extractedType ?? "").lowercased()
        let retry = parseRetryAfter(headers: headers, body: body)

        // 1. 优先根据明确的业务错误特征识别（即使 HTTP 状态码不标准也能精准识别）
        if isRateLimited(statusCode: statusCode, type: normalizedType, message: normalizedMsg, body: normalizedBody) {
            return ClassifiedProviderError(
                category: .rateLimited,
                statusCode: statusCode ?? 429,
                errorType: extractedType ?? "rate_limit_exceeded",
                serverMessage: extractedMsg,
                retryAfter: retry,
                underlying: underlying,
                diagnostics: diagnostics
            )
        }

        if isContextLengthExceeded(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
            return ClassifiedProviderError(
                category: .contextExceeded,

```

```

        statusCode: statusCode ?? 400,
        errorType: extractedType ?? "context_length_exceeded",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

if isQuotaExceeded(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
    return ClassifiedProviderError(
        category: .quotaExhausted,
        statusCode: statusCode ?? 402,
        errorType: extractedType ?? "insufficient_quota",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

if isContentFiltered(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
    return ClassifiedProviderError(
        category: .contentFiltered,
        statusCode: statusCode ?? 400,
        errorType: extractedType ?? "content_filter",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

if isModelNotFound(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
    return ClassifiedProviderError(
        category: .modelNotFound,
        statusCode: statusCode ?? 404,
        errorType: extractedType ?? "model_not_found",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

// 2. 根据标准 HTTP 状态码分类
if let code = statusCode {
    switch code {
    case 400:
        return ClassifiedProviderError(
            category: .invalidRequest,
            statusCode: code,
            errorType: extractedType,
            serverMessage: extractedMsg,
            underlying: underlying,
            diagnostics: diagnostics
        )
    case 401:
        return ClassifiedProviderError(
            category: .authFailure,
            statusCode: code,
            errorType: extractedType ?? "unauthorized",
            serverMessage: extractedMsg,
            underlying: underlying,

```

```
        diagnostics: diagnostics
    )
case 402:
    return ClassifiedProviderError(
        category: .quotaExhausted,
        statusCode: code,
        errorType: extractedType ?? "payment_required",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 403:
    return ClassifiedProviderError(
        category: .accessForbidden,
        statusCode: code,
        errorType: extractedType ?? "forbidden",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 404:
    return ClassifiedProviderError(
        category: .modelNotFound,
        statusCode: code,
        errorType: extractedType ?? "not_found",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 405:
    return ClassifiedProviderError(
        category: .methodNotAllowed,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 408:
    return ClassifiedProviderError(
        category: .gatewayTimeout,
        statusCode: code,
        errorType: extractedType ?? "request_timeout",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 409:
    return ClassifiedProviderError(
        category: .conflict,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 413:
    return ClassifiedProviderError(
        category: .payloadTooLarge,
        statusCode: code,
        errorType: extractedType ?? "payload_too_large",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
```

```
)
case 415:
    return ClassifiedProviderError(
        category: .unsupportedMediaType,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 422:
    return ClassifiedProviderError(
        category: .invalidRequest,
        statusCode: code,
        errorType: extractedType ?? "unprocessable_entity",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 423:
    return ClassifiedProviderError(
        category: .locked,
        statusCode: code,
        errorType: extractedType ?? "locked",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 429:
    return ClassifiedProviderError(
        category: .rateLimited,
        statusCode: code,
        errorType: extractedType ?? "rate_limit_exceeded",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 500:
    return ClassifiedProviderError(
        category: .internalServerError,
        statusCode: code,
        errorType: extractedType ?? "internal_server_error",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 501:
    return ClassifiedProviderError(
        category: .notImplemented,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 502, 520, 521, 523:
    return ClassifiedProviderError(
        category: .badGateway,
        statusCode: code,
        errorType: extractedType ?? "bad_gateway",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
```

```

        diagnostics: diagnostics
    )
}
case 503, 529:
    return ClassifiedProviderError(
        category: .serverOverloaded,
        statusCode: code,
        errorType: extractedType ?? "service_unavailable",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 504, 522, 524:
    return ClassifiedProviderError(
        category: .gatewayTimeout,
        statusCode: code,
        errorType: extractedType ?? "gateway_timeout",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
default:
    if code >= 500 {
        return ClassifiedProviderError(
            category: .badGateway,
            statusCode: code,
            errorType: extractedType,
            serverMessage: extractedMsg,
            retryAfter: retry,
            underlying: underlying,
            diagnostics: diagnostics
        )
    }
}
}

// 3. 检查底层网络/传输错误
if let underlying {
    let desc = underlying.localizedDescription.lowercased()
    if let coreErr = underlying as? CoreError {
        if coreErr.code == .commandTimedOut {
            return ClassifiedProviderError(
                category: .gatewayTimeout,
                statusCode: 408,
                errorType: "command_timed_out",
                serverMessage: coreErr.message,
                retryAfter: retry,
                underlying: underlying,
                diagnostics: diagnostics
            )
        }
        if coreErr.code == .transportLost {
            return ClassifiedProviderError(
                category: .networkFailure,
                statusCode: nil,
                errorType: "transport_lost",
                serverMessage: coreErr.message,
                retryAfter: retry,
                underlying: underlying,
                diagnostics: diagnostics
            )
        }
    }
}
if desc.contains("timed out") || desc.contains("timeout") {

```

```

        return ClassifiedProviderError(
            category: .gatewayTimeout,
            statusCode: 408,
            errorType: "timeout",
            serverMessage: underlying.localizedDescription,
            retryAfter: retry,
            underlying: underlying,
            diagnostics: diagnostics
        )
    }
}

if desc.contains("connection reset") || desc.contains("network connection lost") || desc.contains("cannot connect") {
    return ClassifiedProviderError(
        category: .networkFailure,
        statusCode: nil,
        errorType: "network_error",
        serverMessage: underlying.localizedDescription,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
}
}

// 4. 兜底
return ClassifiedProviderError(
    category: .unknown,
    statusCode: statusCode,
    errorType: extractedType,
    serverMessage: extractedMsg,
    retryAfter: retry,
    underlying: underlying,
    diagnostics: diagnostics
)
}

// MARK: - 辅助特征识别

private static func isContextLengthExceeded(type: String, message: String, body: String) -> Bool {
    type.contains("context_length_exceeded")
    || type.contains("context_window_exceeded")
    || message.contains("context length")
    || message.contains("maximum context length")
    || message.contains("context_length_exceeded")
    || message.contains("prompt is too long")
    || message.contains("tokens exceed")
    || body.contains("context_length_exceeded")
    || (body.contains("maximum context length") && body.contains("tokens"))
}

private static func isQuotaExceeded(type: String, message: String, body: String) -> Bool {
    guard !message.contains("rpm") && !message.contains("tpm") && !body.contains("rpm exhausted") && !body.contains("tpm exhausted")
    ↪ else {
        return false
    }
    return type.contains("insufficient_quota")
        || type.contains("quota_exceeded")
        || type.contains("credit_expired")
        || type.contains("billing_not_active")
        || message.contains("insufficient quota")
        || message.contains("exceeded your current quota")
        || message.contains("quota exceeded")
        || message.contains("balance insufficient")
        || message.contains("额度不足")
        || message.contains("余额不足")
        || message.contains("欠费")
}

```

```

        || body.contains("insufficient_quota")
    }

private static func isContentFiltered(type: String, message: String, body: String) -> Bool {
    type.contains("content_filter")
    || type.contains("sensitive_content")
    || message.contains("content filter")
    || message.contains("safety system")
    || message.contains("moderation")
    || message.contains("合规")
    || message.contains("违规")
    || body.contains("content_filter")
}

private static func isModelNotFound(type: String, message: String, body: String) -> Bool {
    type.contains("model_not_found")
    || type.contains("model_not_exists")
    || message.contains("model does not exist")
    || message.contains("model not found")
    || message.contains("no such model")
    || (type.contains("invalid_request_error") && message.contains("model"))
}

private static func isRateLimited(statusCode: Int?, type: String, message: String, body: String) -> Bool {
    statusCode == 429
    || type.contains("rate_limit")
    || type.contains("rate_limit_exceeded")
    || type.contains("tpm_exhausted")
    || type.contains("rpm_exhausted")
    || message.contains("rate limit")
    || message.contains("too many requests")
    || message.contains("tpm exhausted")
    || message.contains("rpm exhausted")
    || message.contains("rpm limit")
    || message.contains("tpm limit")
    || body.contains("429001")
    || (body.contains("inference") && body.contains("tpm") && body.contains("exhaust"))
    || (body.contains("inference") && body.contains("rpm") && body.contains("exhaust"))
}

private static func parseJSONError(_ body: String?) -> (type: String?, message: String?) {
    guard let body, !body.isEmpty, let data = body.data(using: .utf8) else { return (nil, nil) }
    guard let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any] else { return (nil, nil) }

    if let errorObj = json["error"] as? [String: Any] {
        let type = errorObj["type"] as? String ?? errorObj["code"] as? String
        let message = errorObj["message"] as? String
        return (type, message)
    } else if let errorMsg = json["error"] as? String {
        return (nil, errorMsg)
    } else if let message = json["message"] as? String {
        let code = json["code"] as? String ?? (json["code"] as? Int).map(String.init)
        return (code, message)
    }
    return (nil, nil)
}

private static func parseRetryAfter(headers: [String: String], body: String?) -> Duration? {
    if let value = headers.first(where: { $0.key.caseInsensitiveCompare("Retry-After-Ms") == .orderedSame })?.value.trimmingCharacters(
        ↪ in: .whitespacesAndNewlines), let milliseconds = Double(value), milliseconds >= 0 {
        return .milliseconds(Int(milliseconds))
    }
    if let value = headers.first(where: { $0.key.caseInsensitiveCompare("Retry-After") == .orderedSame })?.value.trimmingCharacters(in:
        ↪ .whitespacesAndNewlines) {
        if let seconds = Double(value), seconds >= 0 { return .milliseconds(Int(seconds * 1_000)) }
    }
}

```

```

        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "en_US_POSIX")
        formatter.timeZone = TimeZone(secondsFromGMT: 0)
        formatter.dateFormat = "EEE', ' dd MMM yyyy HH':'mm':'ss z"
        if let date = formatter.date(from: value) {
            return .milliseconds(max(0, Int(date.timeIntervalSinceNow * 1_000)))
        }
    }
}
return nil
}
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderConfig.swift

```

import Foundation
import LingXiProtocol

/// Provider 实例与已解析端点必须原子组装，endpoint 是运行时唯一真相。
public struct ModelRuntimeAssembly: Sendable {
    public let provider: any ModelProvider
    public let endpoint: ResolvedModelEndpoint
    public var modelID: ModelID { endpoint.modelID }
    public var contextProfile: ModelContextProfile { endpoint.contextProfile }

    public init(provider: any ModelProvider, modelID: ModelID, contextProfile: ModelContextProfile = ModelContextProfile(), endpoint:
        ↳ ResolvedModelEndpoint? = nil) {
        self.provider = provider
        self.endpoint = endpoint ?? ResolvedModelEndpoint(providerID: "default", modelID: modelID, baseURL: nil, wireProtocol: .
            ↳ chatCompletions, contextProfile: contextProfile)
    }

    public static let unavailable = ModelRuntimeAssembly(provider: UnavailableProvider(), modelID: ModelID(""))
}

private struct UnavailableProvider: ModelProvider {
    func stream(_ request: ModelRequest) async throws -> AsyncThrowingStream<ModelEvent, Error> {
        throw CoreError(code: .provider, message: "未配置模型 Provider")
    }
}

/// Provider 连接配置（仅存在于 Core 内部，不进入协议层）。
public struct ProviderConfig: Sendable {
    public let baseURL: URL
    public let authentication: ProviderAuthentication
    public let model: String
    public let wireProtocol: ModelWireProtocol
    public let diagnosticsEnabled: Bool
    public let performanceDiagnosticsEnabled: Bool
    public let remoteStateEnabled: Bool
    public let maxOutputTokens: Int?
    public let requiredHeaders: [String: String]
    public let parallelToolCalling: Bool?

    public init(
        baseURL: URL,
        apiKey: String?,
        model: String,
        wireProtocol: ModelWireProtocol = .chatCompletions,
        diagnosticsEnabled: Bool = false,
        performanceDiagnosticsEnabled: Bool = false,
        remoteStateEnabled: Bool = false,
        maxOutputTokens: Int? = nil,
        requiredHeaders: [String: String] = [:],
        parallelToolCalling: Bool? = true
    ) {

```

```

        self.baseURL = baseURL
        authentication = apiKey.map(ProviderAuthentication.bearer) ?? .none
        self.model = model
        self.wireProtocol = wireProtocol
        self.diagnosticsEnabled = diagnosticsEnabled
        self.performanceDiagnosticsEnabled = performanceDiagnosticsEnabled
        self.remoteStateEnabled = remoteStateEnabled
        self.maxOutputTokens = maxOutputTokens
        self.requiredHeaders = requiredHeaders
        self.parallelToolCalling = parallelToolCalling
    }

    public init(
        baseURL: URL,
        authentication: ProviderAuthentication,
        model: String,
        wireProtocol: ModelWireProtocol,
        diagnosticsEnabled: Bool = false,
        performanceDiagnosticsEnabled: Bool = false,
        remoteStateEnabled: Bool = false,
        maxOutputTokens: Int? = nil,
        requiredHeaders: [String: String] = [:],
        parallelToolCalling: Bool? = true
    ) {
        self.baseURL = baseURL
        self.authentication = authentication
        self.model = model
        self.wireProtocol = wireProtocol
        self.diagnosticsEnabled = diagnosticsEnabled
        self.performanceDiagnosticsEnabled = performanceDiagnosticsEnabled
        self.remoteStateEnabled = remoteStateEnabled
        self.maxOutputTokens = maxOutputTokens
        self.requiredHeaders = requiredHeaders
        self.parallelToolCalling = parallelToolCalling
    }

    /// OpenAI-compatible chat completions 端点。
    /// Base URL 已包含最终兼容层级（如 .../chat/completions）时不再追加，避免 /v1/v1。
    public var chatCompletionsURL: URL {
        let trimmedString = baseURL.absoluteString.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
        if trimmedString.hasSuffix("/chat/completions") {
            return baseURL
        }
        return URL(string: trimmedString + "/chat/completions") ?? baseURL
    }

    public var responsesURL: URL {
        let trimmedString = baseURL.absoluteString.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
        if trimmedString.hasSuffix("/responses") { return baseURL }
        return URL(string: trimmedString + "/responses") ?? baseURL
    }

    public var anthropicMessagesURL: URL {
        let trimmedString = baseURL.absoluteString.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
        if trimmedString.hasSuffix("/v1/messages") || trimmedString.hasSuffix("/messages") { return baseURL }
        return URL(string: trimmedString + "/v1/messages") ?? baseURL
    }
}

public enum ProviderAuthentication: Sendable, Equatable {
    case none
    case bearer(String)
    case header(name: String, value: String)
}

```

```

/// Wire protocol is selected by the resolved endpoint, not by a Provider brand.
public enum ModelWireProtocol: String, Sendable, Equatable, Codable {
    case chatCompletions
    case responses
    case anthropicMessages
}

public struct ResolvedModelEndpoint: Sendable, Equatable {
    public let providerID: String
    public let productID: String
    public let endpointID: String?
    public let accountID: String?
    public let profileID: String?
    public let modelID: ModelID
    public let baseURL: URL?
    public let wireProtocol: ModelWireProtocol
    public let contextProfile: ModelContextProfile
    public let capabilities: ModelCapabilities
    public let rateLimits: ProviderRateLimits

    public init(providerID: String, productID: String? = nil, endpointID: String? = nil, accountID: String? = nil, profileID: String? = nil,
        ↪ modelID: ModelID, baseURL: URL?, wireProtocol: ModelWireProtocol, contextProfile: ModelContextProfile = ModelContextProfile(),
        ↪ capabilities: ModelCapabilities = ModelCapabilities(), rateLimits: ProviderRateLimits = ProviderRateLimits()) {
        self.providerID = providerID
        self.productID = productID ?? providerID
        self.endpointID = endpointID
        self.accountID = accountID
        self.profileID = profileID
        self.modelID = modelID
        self.baseURL = baseURL
        self.wireProtocol = wireProtocol
        self.contextProfile = contextProfile
        self.capabilities = capabilities
        self.rateLimits = rateLimits
    }
}

public struct ModelCapabilities: Codable, Sendable, Equatable {
    public let toolCalling: Bool?
    public let parallelToolCalling: Bool?
    public let reasoning: Bool?
    public let vision: Bool?
    public let structuredOutput: Bool?
    public let reasoningCapability: ReasoningCapability?

    public init(toolCalling: Bool? = nil, parallelToolCalling: Bool? = nil, reasoning: Bool? = nil, vision: Bool? = nil, structuredOutput:
        ↪ Bool? = nil, reasoningCapability: ReasoningCapability? = nil) {
        self.toolCalling = toolCalling
        self.parallelToolCalling = parallelToolCalling
        self.reasoning = reasoning
        self.vision = vision
        self.structuredOutput = structuredOutput
        self.reasoningCapability = reasoningCapability
    }
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderHTTPTransport.swift

```

import Foundation
#if canImport(FoundationNetworking)
import FoundationNetworking
#endif
import LingXiProtocol

public struct ProviderHTTPRequestContext: Sendable, Equatable {

```

```

public let wireProtocol: ModelWireProtocol
public let model: String
public let requestID: ModelRequestID
public let executionID: AgentRunID?
public let step: Int

public init(wireProtocol: ModelWireProtocol, model: String, requestID: ModelRequestID = ModelRequestID(), executionID: AgentRunID?, step
↵ : Int) {
    self.wireProtocol = wireProtocol
    self.model = model
    self.requestID = requestID
    self.executionID = executionID
    self.step = step
}
}

public struct ProviderHTTPResponse: Sendable {
    public let statusCode: Int
    public let headers: [String: String]
    public let body: AsyncThrowingStream<Data, Error>

    public init(statusCode: Int, headers: [String: String] = [:], body: AsyncThrowingStream<Data, Error>) {
        self.statusCode = statusCode
        self.headers = headers
        self.body = body
    }

    public func header(_ name: String) -> String? {
        headers.first { $0.key.caseInsensitiveCompare(name) == .orderedSame }?.value
    }
}

public protocol ProviderHTTPTransport: Sendable {
    func send(_ request: URLRequest, context: ProviderHTTPRequestContext) async throws -> ProviderHTTPResponse
}

public struct URLSessionProviderHTTPTransport: ProviderHTTPTransport {
    public static let sharedSession: URLSession = {
        let config = URLSessionConfiguration.default
        config.httpMaximumConnectionsPerHost = 16
        config.timeoutIntervalForRequest = 45
        config.timeoutIntervalForResource = 300
        config.waitsForConnectivity = false
        return URLSession(configuration: config)
    }()

    private let session: URLSession

    public init(session: URLSession = URLSessionProviderHTTPTransport.sharedSession) {
        self.session = session
    }

    public func send(_ request: URLRequest, context: ProviderHTTPRequestContext) async throws -> ProviderHTTPResponse {
        let bytes: URLSession.AsyncBytes
        let response: URLResponse
        do {
            (bytes, response) = try await session.bytes(for: request)
        } catch is CancellationError {
            throw CancellationError()
        } catch let error as URLError where error.code == .timedOut {
            throw CoreError(code: .commandTimedOut, message: "Provider request timed out")
        } catch let error as URLError where error.code == .cancelled {
            throw CancellationError()
        } catch {
            if Task.isCancelled { throw CancellationError() }
        }
    }
}

```

```

        let detail = (error as? URLError)?.localizedDescription ?? error.localizedDescription
        throw CoreError(code: .transportLost, message: "Provider transport failed: \(detail)")
    }
    guard let http = response as? HTTPURLResponse else {
        throw CoreError(code: .provider, message: "Provider 返回非 HTTP 响应")
    }
    let body = AsyncThrowingStream<Data, Error> { continuation in
        let pump = Task {
            do {
                var chunk = Data()
                for try await byte in bytes {
                    if Task.isCancelled { throw CancellationError() }
                    chunk.append(byte)
                    if byte == 0x0A || chunk.count >= 4_096 {
                        continuation.yield(chunk)
                        chunk.removeAll(keepingCapacity: true)
                    }
                }
                if !chunk.isEmpty {
                    continuation.yield(chunk)
                }
                continuation.finish()
            } catch {
                continuation.finish(throwing: error)
            }
        }
        continuation.onTermination = { @Sendable _ in
            pump.cancel()
            bytes.task.cancel()
        }
    }
    return ProviderHTTPResponse(
        statusCode: http.statusCode,
        headers: Dictionary(uniqueKeysWithValues: http.allHeaderFields.compactMap { key, value in
            guard let key = key as? String else { return nil }
            return (key, String(describing: value))
        }),
        body: body
    )
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderActivityRegistry.swift

```

import Foundation
import LingXiProtocol

public actor ProviderActivityRegistry {
    public static let shared = ProviderActivityRegistry()

    private var activities: [String: ProviderActivitySnapshot] = [:] // providerRequestID -> snapshot
    private var cancelledRequestIDs: Set<String> = []
    private var cancelledRunIDs: Set<AgentRunID> = []

    public init() {}

    @discardableResult
    public func record(
        sessionID: SessionID,
        runID: AgentRunID?,
        providerRequestID: String,
        state: ProviderActivityState,
        model: String? = nil,
        detail: String? = nil,
        statusCode: Int? = nil
    )

```

```

) -> ProviderActivitySnapshot {
    let effectiveState: ProviderActivityState
    if let runID, cancelledRunIDs.contains(runID) {
        effectiveState = .cancelled
    } else if cancelledRequestIDs.contains(providerRequestID) && !state.isTerminal {
        effectiveState = .cancelled
    } else {
        effectiveState = state
    }
    let snapshot = ProviderActivitySnapshot(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: effectiveState,
        model: model,
        detail: detail,
        statusCode: statusCode,
        updatedAt: .now
    )
    activities[providerRequestID] = snapshot
    return snapshot
}

@discardableResult
public func cancel(providerRequestID: String) -> ProviderActivitySnapshot? {
    guard !providerRequestID.isEmpty else { return nil }
    cancelledRequestIDs.insert(providerRequestID)
    guard var snapshot = activities[providerRequestID] else { return nil }
    snapshot = ProviderActivitySnapshot(
        sessionID: snapshot.sessionID,
        runID: snapshot.runID,
        providerRequestID: snapshot.providerRequestID,
        state: .cancelled,
        model: snapshot.model,
        detail: snapshot.detail,
        statusCode: snapshot.statusCode,
        updatedAt: .now
    )
    activities[providerRequestID] = snapshot
    return snapshot
}

@discardableResult
public func cancelRun(_ runID: AgentRunID) -> [ProviderActivitySnapshot] {
    cancelledRunIDs.insert(runID)
    var updated: [ProviderActivitySnapshot] = []
    for (id, snapshot) in activities where snapshot.runID == runID && !snapshot.state.isTerminal {
        if !id.isEmpty {
            cancelledRequestIDs.insert(id)
        }
        let newSnapshot = ProviderActivitySnapshot(
            sessionID: snapshot.sessionID,
            runID: snapshot.runID,
            providerRequestID: snapshot.providerRequestID,
            state: .cancelled,
            model: snapshot.model,
            detail: snapshot.detail,
            statusCode: snapshot.statusCode,
            updatedAt: .now
        )
        activities[id] = newSnapshot
        updated.append(newSnapshot)
    }
    return updated
}

```

```

public func isCancelled(providerRequestID: String, runID: AgentRunID?) -> Bool {
    if !providerRequestID.isEmpty && cancelledRequestIDs.contains(providerRequestID) { return true }
    if let runID, cancelledRunIDs.contains(runID) { return true }
    return false
}

public func isRunCancelled(_ runID: AgentRunID) -> Bool {
    cancelledRunIDs.contains(runID)
}

public func activeActivities(for sessionID: SessionID) -> [ProviderActivitySnapshot] {
    activities.values.filter { $0.sessionID == sessionID && !$0.state.isTerminal }
}

public func latestActivity(for runID: AgentRunID) -> ProviderActivitySnapshot? {
    activities.values.filter { $0.runID == runID }.max(by: { $0.updatedAt < $1.updatedAt })
}

public func reset() {
    activities.removeAll()
    cancelledRequestIDs.removeAll()
    cancelledRunIDs.removeAll()
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderProvenance.swift

```

import Foundation
import LingXiProtocol

/// Provider 原生调用身份只存在于 Model Adapter 边界，不能成为 LingXi ToolCallID。
public struct ProviderToolCallReference: Sendable, Equatable, Codable {
    public let wire: ModelWireProtocol
    public let domainCallID: ToolCallID
    public let externalCallID: String
    public let externalItemID: String?

    public init(wire: ModelWireProtocol, domainCallID: ToolCallID, externalCallID: String, externalItemID: String? = nil) {
        self.wire = wire
        self.domainCallID = domainCallID
        self.externalCallID = externalCallID
        self.externalItemID = externalItemID
    }
}

public enum ProviderContinuationItem: Sendable, Equatable, Codable {
    /// 原始 Provider item JSON; Core 不解释或改写其中的 opaque content。
    case opaque(Data)
    case toolCall(ToolCallID)
}

public struct ProviderContinuation: Sendable, Equatable, Codable {
    public let requestID: ModelRequestID
    public let executionID: AgentRunID?
    public let wire: ModelWireProtocol
    public let responseID: String?
    public let references: [ProviderToolCallReference]
    public let orderedItems: [ProviderContinuationItem]

    public init(requestID: ModelRequestID, executionID: AgentRunID?, wire: ModelWireProtocol, responseID: String? = nil, references: [
        ⇐ ProviderToolCallReference], orderedItems: [ProviderContinuationItem] = []) {
        self.requestID = requestID
        self.executionID = executionID
        self.wire = wire
    }
}

```

```

        self.responseID = responseID
        self.references = references
        self.orderedItems = orderedItems
    }

    public func externalCallID(for domainCallID: ToolCallID) -> String? {
        references.first { $0.domainCallID == domainCallID }?.externalCallID
    }
}

/// Entries are keyed by one model request, so concurrent Main/Child decoders cannot share correlation state.
public actor ProviderProvenanceStore {
    private var entries: [ModelRequestID: ProviderContinuation] = [:]
    private var referencesByDomainID: [ToolCallID: ProviderToolCallReference] = [:]
    private let directory: URL

    public init(directory: URL? = nil) {
        self.directory = directory ?? FileManager.default.temporaryDirectory.appendingPathComponent("lingxi-provider-provenance-\(UUID().
        ↪ uuidString)", isDirectory: true)
        try? FileManager.default.createDirectory(at: self.directory, withIntermediateDirectories: true)
        try? FileManager.default.setAttributes([.posixPermissions: 0o700], ofItemAtPath: self.directory.path)
    }

    public func continuation(for requestID: ModelRequestID?, wire: ModelWireProtocol) throws -> ProviderContinuation? {
        guard let requestID else { return nil }
        let url = file(requestID)
        let value: ProviderContinuation?
        if let cached = entries[requestID] { value = cached }
        else if FileManager.default.fileExists(atPath: url.path) { value = try JSONDecoder().decode(ProviderContinuation.self, from: Data(
        ↪ contentsOf: url)) }
        else { value = nil }
        guard let value, value.wire == wire else { return nil }
        entries[requestID] = value
        for ref in value.references { referencesByDomainID[ref.domainCallID] = ref }
        return value
    }

    public func resolveContinuation(for requestID: ModelRequestID?, wire: ModelWireProtocol) throws -> ProviderContinuation? {
        let base = try continuation(for: requestID, wire: wire)
        var combined = base?.references ?? []
        for (domainID, ref) in referencesByDomainID where ref.wire == wire {
            if !combined.contains(where: { $0.domainCallID == domainID }) {
                combined.append(ref)
            }
        }
        if let base {
            return ProviderContinuation(requestID: base.requestID, executionID: base.executionID, wire: base.wire, responseID: base.
            ↪ responseID, references: combined, orderedItems: base.orderedItems)
        } else if !combined.isEmpty {
            return ProviderContinuation(requestID: requestID ?? ModelRequestID(), executionID: nil, wire: wire, responseID: nil, references:
            ↪ combined, orderedItems: [])
        }
        return nil
    }

    public func record(_ continuation: ProviderContinuation) throws {
        let url = file(continuation.requestID)
        try JSONEncoder().encode(continuation).write(to: url, options: .atomic)
        try FileManager.default.setAttributes([.posixPermissions: 0o600], ofItemAtPath: url.path)
        entries[continuation.requestID] = continuation
        for ref in continuation.references {
            referencesByDomainID[ref.domainCallID] = ref
        }
    }
}

```

```

public func remove(_ executionID: AgentRunID) {
    entries = entries.filter { $0.value.executionID != executionID }
    for url in files() {
        guard let data = try? Data(contentsOf: url), let value = try? JSONDecoder().decode(ProviderContinuation.self, from: data), value
        ↪ .executionID == executionID else { continue }
        try? FileManager.default.removeItem(at: url)
    }
}

private func files() -> [URL] { (try? FileManager.default.contentsOfDirectory(at: directory, includingPropertiesForKeys: nil)) ?? [] }
private func file(_ requestID: ModelRequestID) -> URL { directory.appendingPathComponent(sha256Hex(requestID.rawValue) + ".json") }
}

```

// File: Sources/LingXiCore/Modules/Model/CanonicalCachePlan.swift

```

import Foundation
import LingXiProtocol

```

/// 跨 Provider、跨 Model、跨 Protocol 的统一缓存结构计划。
 /// 在任何 Provider wire encoding 之前建立，各 Provider Adapter 只能将该 Plan 序列化为对应协议，
 /// 严禁 Adapter 自行决定哪些内容是 Stable 还是 Volatile。

```

public struct CanonicalCachePlan: Sendable, Equatable {
    public struct EpochIdentity: Sendable, Equatable {
        public let epoch: Int
        public let reason: String

        public init(epoch: Int, reason: String = "initial") {
            self.epoch = epoch
            self.reason = reason
        }
    }
}

```

```

public struct ImmutableBase: Sendable, Equatable {
    public let systemPrompt: String?
    public let developerPrompt: String?
    public let coreTools: [ToolDefinition]
    public let stablePolicy: String?

    public init(
        systemPrompt: String? = nil,
        developerPrompt: String? = nil,
        coreTools: [ToolDefinition] = [],
        stablePolicy: String? = nil
    ) {
        self.systemPrompt = systemPrompt
        self.developerPrompt = developerPrompt
        self.coreTools = coreTools
        self.stablePolicy = stablePolicy
    }
}

```

```

public struct AppendOnlyContext: Sendable, Equatable {
    public let dynamicTools: [ToolDefinition]
    public let messages: [ModelMessage]
    public let skillActivations: [String]

    public init(
        dynamicTools: [ToolDefinition] = [],
        messages: [ModelMessage] = [],
        skillActivations: [String] = []
    ) {
        self.dynamicTools = dynamicTools
        self.messages = messages
        self.skillActivations = skillActivations
    }
}

```

```

    }

    public struct VolatileTail: Sendable, Equatable {
        public let currentTurnState: String?
        public let ephemeralNotes: String?

        public init(currentTurnState: String? = nil, ephemeralNotes: String? = nil) {
            self.currentTurnState = currentTurnState
            self.ephemeralNotes = ephemeralNotes
        }
    }

    public let epochIdentity: EpochIdentity
    public let immutableBase: ImmutableBase
    public let appendOnlyContext: AppendOnlyContext
    public let volatileTail: VolatileTail
    public let structuralHealth: ClientStructuralCacheHealth
    public let capabilities: ProviderCacheCapabilities?

    public init(
        epochIdentity: EpochIdentity,
        immutableBase: ImmutableBase,
        appendOnlyContext: AppendOnlyContext,
        volatileTail: VolatileTail = VolatileTail(),
        structuralHealth: ClientStructuralCacheHealth,
        capabilities: ProviderCacheCapabilities? = nil
    ) {
        self.epochIdentity = epochIdentity
        self.immutableBase = immutableBase
        self.appendOnlyContext = appendOnlyContext
        self.volatileTail = volatileTail
        self.structuralHealth = structuralHealth
        self.capabilities = capabilities
    }
}

```

// File: Sources/LingXiCore/Modules/Model/SSEDecoder.swift

```

import Foundation

/// SSE 行缓冲解码器。
/// 网络字节按任意大小的块喂入，按完整行输出；
/// 不假设一个 TCP chunk 对应一个 SSE event，处理 \n 与 \r\n。
public struct SSEDecoder {
    private var buffer: [UInt8] = []

    public init() {}

    /// 喂入任意网络块，返回其中已完整到达的行（不含行终止符）。
    public mutating func feed(_ data: Data) -> [String] {
        data.withUnsafeBytes { raw in
            buffer.append(contentsOf: raw.bindMemory(to: UInt8.self))
        }
        return extractLines()
    }

    /// 流结束时调用：输出残留的最后一行（SSE 允许末行无换行）。
    public mutating func flushPending() -> [String] {
        guard !buffer.isEmpty else { return [] }
        let line = String(decoding: buffer, as: UTF8.self)
        buffer.removeAll()
        return [line]
    }

    private mutating func extractLines() -> [String] {

```

```
var lines: [String] = []
var lineStart = buffer.startIndex
var index = buffer.startIndex

while index < buffer.endIndex {
    guard buffer[index] == UInt8(ascii: "\n") else {
        index += 1
        continue
    }
    var end = index
    if end > lineStart, buffer[end - 1] == UInt8(ascii: "\r") {
        end -= 1
    }
    lines.append(String(decoding: buffer[lineStart..
```